



PicoBot — Robot Design

Mechanical, electronic and control-software design of the PicoBot educational
mobile manipulator · Version 4.0

Table of Contents

About this document.....	2
1. Requirements.....	3
1.1 What the robot must do.....	3
1.2 Technical requirements.....	3
1.3 Design approach.....	4
2. System overview.....	5
2.1 The robot.....	5
2.2 Block diagram.....	7
2.3 Design principles.....	7
3. Mechanical design.....	8
3.1 Chassis.....	8
3.2 Placement of the components.....	9
3.3 Drivetrain: why mecanum wheels.....	9
3.4 How a mecanum wheel works.....	10
3.5 Kinematics.....	12
3.6 Motors and practical limits of the drive.....	13
3.7 Robot arm.....	13
3.8 Mechanical robustness.....	15
4. Electronic design.....	15
4.1 Architecture and interfaces.....	15
4.2 Controller: Raspberry Pi Pico 2 W.....	16
4.3 Motor driver: Waveshare Pico-Motor-Driver.....	17
4.4 Servo driver: PCA9685 16-channel board.....	20

4.5 Battery.....	21
4.6 Power distribution.....	22
4.7 Communication buses.....	24
4.8 Sensors.....	25
4.9 Pin map.....	29
5. Control software on the robot.....	30
5.1 Platform.....	30
5.2 Software layers.....	30
5.3 Motion control.....	31
5.4 User interface and communication.....	32
5.5 Programs for the four reference tasks.....	32
5.6 Safety functions in software.....	33
6. Safety design.....	34
7. Verification.....	34
7.1 Requirements check.....	34
7.2 Findings of the first tests and the design response.....	35
8. Known limitations and recommended improvements.....	36
9. Values to measure on a built robot.....	36
10. Hardware revisions.....	37
Appendix A. Bill of materials (Rev3).....	38
Appendix B. Wiring list (Rev3).....	38
Appendix C. References.....	40
Credits and licence.....	40

About this document

PicoBot is a small, open educational robot: a four-wheel **mecanum** base with a **robot arm**, controlled by a **Raspberry Pi Pico 2 W** and programmed in **MicroPython**. It was developed in the Erasmus+ project **ROBO STEAM ACADEMY** (KA220-VET-7CF4F308, 2024–2027), with the design work led by the Institute of Information and Communication Technologies of the Bulgarian Academy of Sciences, for teaching robotics, electronics and programming in vocational and secondary schools and for robot competitions. The design, the documentation and the program code are open: anybody may build, use and adapt the robot.

This document describes the **design of the robot**. For every part of the robot it answers four questions:

1. **What is it?** — the part, its main technical data and its job on the robot.
2. **Why was it chosen?** — the requirement it meets and the reasons it fits better than the usual alternatives.

3. **How does it work?** — the physical or electrical principle, explained to the level needed to understand, repair and extend the robot.
4. **How does it interact with the rest of the robot?** — power, signals, buses, software.

It is written for teachers, students, technicians, makers and engineers — for anybody who wants to **build, repair, adapt or improve PicoBot**. The parts list and the complete wiring list are in Appendices A and B. Values that are **calculated** from data-sheet figures rather than measured on PicoBot are marked “**estimate**”; section 9 lists what you can measure on your own robot.

1. Requirements

1.1 What the robot must do

PicoBot is a platform for learning **electronics, programming and the theory of mobile manipulators and robot arms** with ready-made programs and libraries in **Python** (MicroPython). It is designed to carry out **four reference tasks**, which are also the tasks of the PicoBot competitions:

#	Task	Control	What the robot must be able to do
1	Line tracking with number of laps	programm atic	follow a line; detect a perpendicular marker line; count laps; stop after the n-th lap
2	Manipulation of an object with the arm	programm atic	follow a straight line (at least 1 m) to a marker; turn the arm sideways to a set angle (e.g. 45°); grip and lift an object; turn to 45° on the other side; lower, release; arm back to transport position; drive 10 cm back; turn 180° on the spot; find the line and return to the start marker
3	Master control of a mobile robot	manual	be driven by a person, using the omnidirectional movement of the mecanum wheels : pass obstacles diagonally; park in “garage cells” that can be entered only forward or only sideways
4	Separate garbage collection	manual	pick up coloured foam objects from the field, carry them and drop them into “garbage containers”

1.2 Technical requirements

The four tasks, the users (vocational and secondary schools) and the use in competitions lead to these technical requirements. Every design decision in sections 3–6 refers back to them.

ID	Requirement	Reason
R1	Controller: Raspberry Pi Pico W (or a compatible successor), programmed in Python (MicroPython)	easy, widely taught language; low-cost controller with Wi-Fi
R2	Omnidirectional drive : forward/backward, sideways, diagonal, rotation on the spot — with mecanum wheels	task 3
R3	Robot arm with a gripper that can turn sideways (at least $\pm 45^\circ$), lift/lower and grip small objects	tasks 2 and 4

ID	Requirement	Reason
R4	Line sensing good enough to follow a line, find it again after a turn, and detect a perpendicular marker	tasks 1 and 2
R5	Distance sensing in front of the robot (obstacles, approach to objects)	task 3, autonomous tasks
R6	Manual remote control without extra hardware, from a phone, tablet or PC	tasks 3 and 4
R7	Programmable by students at several levels: simple named moves for beginners, direct access to every motor and servo for advanced users	learning programming
R8	Battery operation for a whole lesson or competition round; rechargeable cells available in every country	school use
R9	Assembly by teachers and students without special skills or tools ; minimal soldering; parts replaceable	school use
R10	Robust and repairable : few mechanical problems in long use; a fault is repaired by swapping a part	school use
R11	Safe for students: low voltage, protected against the typical mistakes (reversed wiring, blocked servo, lost control link)	school use
R12	Competition limits : at most 30 × 30 × 30 cm and 2 kg , built around a Raspberry Pi Pico W or compatible	competition rules
R13	Low cost and standard parts , available from normal electronics shops	school budgets, replication
R14	Open design : documentation and code free to use, adapt and extend (spare inputs, spare PWM channels)	open educational resource

1.3 Design approach

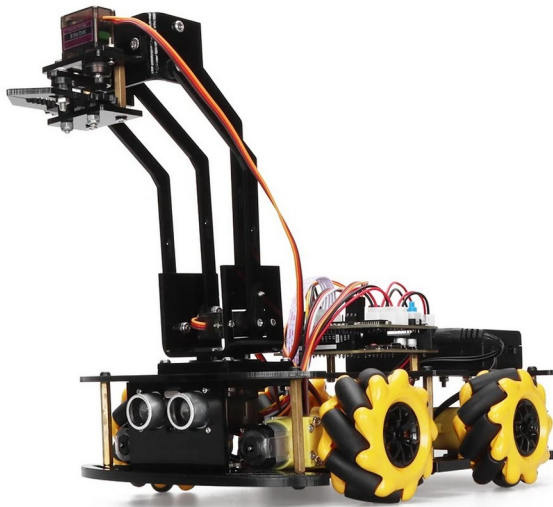
PicoBot is **not designed from zero**. It combines:

- a **commercial mechanical kit** (chassis, four mecanum wheels with gear motors, a three-servo arm, an ultrasonic sensor, a battery holder), sold with Arduino electronics; and
- **new control electronics and software**: the kit's Arduino boards and its 3-channel line sensor are **not used**; they are replaced by a Raspberry Pi Pico 2 W on a motor-driver base board, a separate servo driver, a dedicated 5 V servo supply, a 5-channel line sensor and MicroPython software.

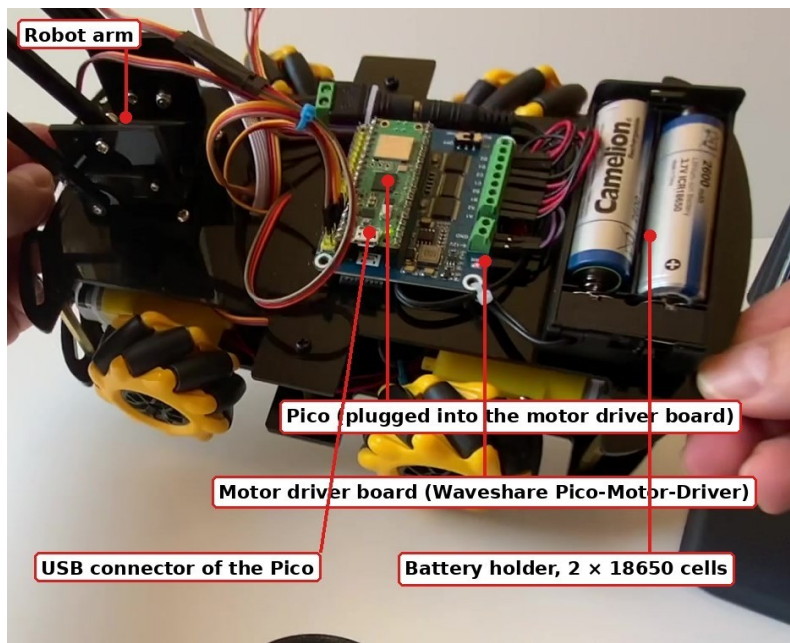
This approach meets R2, R3, R9 and R13 at the same time. A mecanum chassis with an arm is mechanically the most demanding part of such a robot (wheel geometry, roller quality, arm linkages); a mass-produced kit delivers it at a fraction of the cost and time of a custom chassis, and every school can buy the same kit. The **educational value** lies in the parts that students can understand and program — the controller, the drivers, the sensors, the power system and the software — and these are exactly the parts that were redesigned.

2. System overview

2.1 The robot



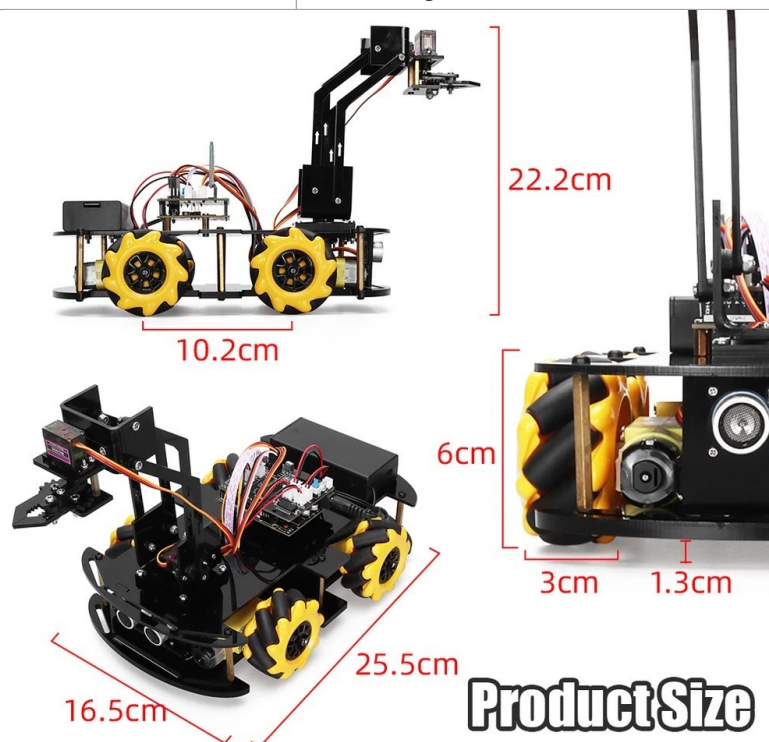
The mechanical kit PicoBot is built from, with the arm raised (photo: kit seller). PicoBot keeps the mechanics shown here; the kit's Arduino boards are replaced by the electronics of section 4.



PicoBot, top deck from behind: the Raspberry Pi Pico 2 W plugged into the Waveshare motor driver board, next to the battery holder with two 18650 cells.

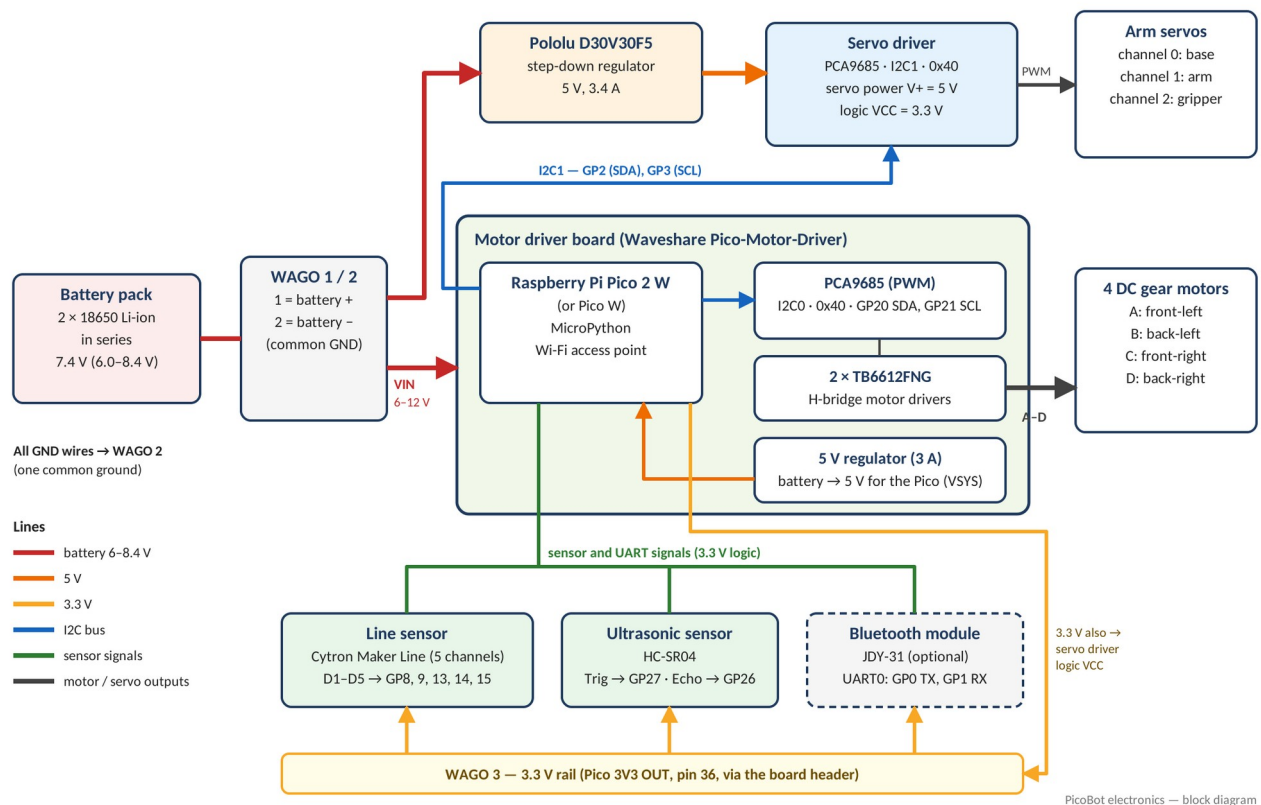
Type	educational omnidirectional mobile manipulator : four-wheel mecanum base + 2-joint arm with gripper
Controller	Raspberry Pi Pico 2 W (RP2350, 2 × 150 MHz, 520 KB SRAM, 4 MB flash, Wi-Fi + Bluetooth); the older Pico W fits and runs the same programs
Programming	MicroPython , Thonny editor; programs on https://github.com/robosteamdev

Drive	4 DC gear motors (TT type) with mecanum wheels $\varnothing \approx 6$ cm; WAVESHARE PICO-Motor-Driver (PCA9685 PWM chip + 2 $\times$ TB6612FNG H-bridges)
Arm	3 micro servos: base (turns the arm), arm (lifts it), gripper ; separate PCA9685 servo driver; own 5 V supply (Pololu D30V30F5, 3.4 A)
Sensors	Cytron Maker Line 5-channel infrared line sensor; HC-SR04 ultrasonic distance sensor
Communication	Wi-Fi access point on the robot (web page at http://192.168.4.1/); optional JDY-31 Bluetooth module
Power	2 $\times$ 18650 Li-ion cells in series: 7.4 V nominal, 8.4 V full, used down to 7.0 V
Size	25.5 $\times$ 16.5 cm, 22.2 cm high with the arm raised; ground clearance 1.3 cm
Mass	about 1 kg with cells (estimate ; limit of R12: 2 kg; see section 9)



Main dimensions: wheelbase 10.2 cm, wheel $\varnothing$ about 6 cm and width 3 cm, ground clearance 1.3 cm, overall 25.5 $\times$ 16.5 $\times$ 22.2 cm (photo: kit seller).

2.2 Block diagram



PicoBot electronics: three power branches (red battery voltage, orange 5 V, yellow 3.3 V), two I2C buses (blue) and the sensor signals (green).

The robot has **one controller** (the Pico), **two “muscle” drivers** that the controller commands over I2C — the motor driver for the wheels and the servo driver for the arm — and **two sensors** read directly by the Pico’s pins. All parts share **one common ground**, the WAGO connector 2.

2.3 Design principles

Seven principles run through the whole design. They are the short answer to “why is PicoBot built this way?”:

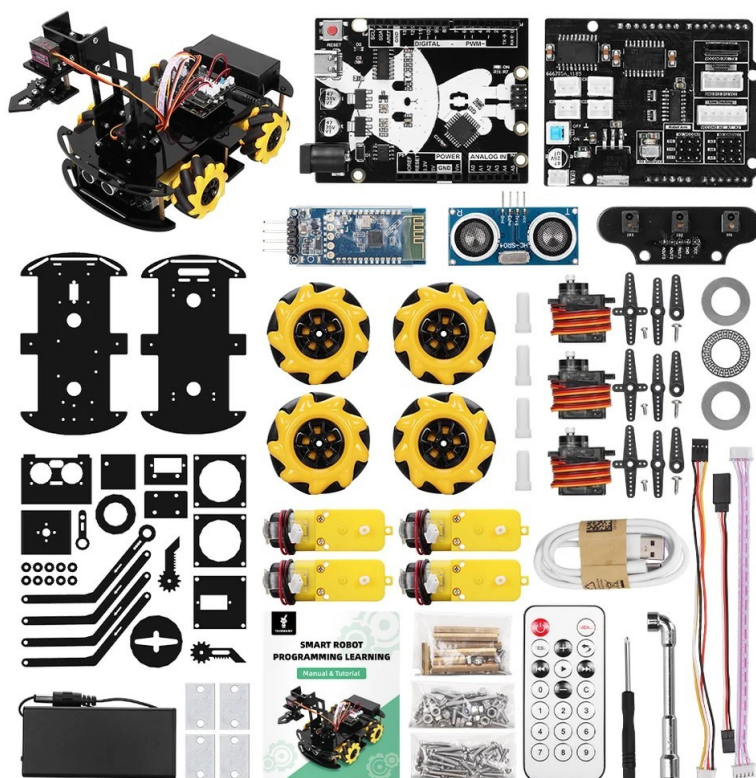
#	Principle	How it is realised	Requirements
P1	Separate power branches for motors, logic and servos	the Pico gets its own regulated 5 V; the servos get their own 5 V regulator; motors run from the battery through the H-bridges	R8, R11
P2	3.3 V logic everywhere	all sensors and the servo driver logic are powered from 3.3 V, so every signal that reaches a Pico pin is 3.3 V	R11, R1
P3	Off-load timing-critical work to dedicated chips	the Pico never makes PWM for motors or servos itself; two PCA9685 chips do it in hardware	R7, R14
P4	Two separate I2C buses	wheels on I2C0, arm on I2C1 — no address conflict, no interference, faults stay local	R9, R10

#	Principle	How it is realised	Requirements
P5	No soldering for the users	plug-in Pico, screw terminals, WAGO lever connectors, DuPont jumper wires; only the regulator's header pins are soldered once by a teacher	R9, R13
P6	Removable mounting	boards fixed with 3M Command strips, so every board can be taken off and replaced without tools	R9, R10
P7	Layered software	from single-register access up to named moves (<code>goForward()</code> , <code>moveRight()</code>), so beginners and advanced users work on the same robot	R7

3. Mechanical design

3.1 Chassis

The chassis is the **Tscinbuny 4WD mecanum robot-arm car kit** (model ZYC0072, sold for Arduino). It consists of two black acrylic plates held apart by brass standoffs, four yellow DC gear motors with mecanum wheels on the lower plate, a three-servo arm at the front of the upper plate, an ultrasonic sensor at the front, and a holder for two 18650 cells at the back.



Contents of the kit (photo: kit seller). PicoBot uses the two chassis plates, the four mecanum wheels and gear motors, the three servos with the arm parts, the ultrasonic sensor, the Bluetooth module (optional) and the battery holder. The Arduino board, its expansion board, the 3-channel line sensor and the infrared remote control are not used.

Why this kit. It is one of very few low-cost kits that combine a **mecanum base** (R2) and a **robot arm with a gripper** (R3) on one chassis, so a single purchase gives the complete mechanics of a mobile manipulator. Its

standard parts — TT gear motors, 9-gram micro servos, an HC-SR04 sensor, an 18650 holder — are sold everywhere and can be replaced one by one (R10, R13). The two-deck layout leaves room for the added electronics: the **lower deck** carries the parts that are wired once and then left alone (servo driver, 5 V regulator, WAGO power connectors), the **upper deck** carries the parts that must be reachable — the Pico with its USB connector and the battery holder with its switch.

Size. With the arm raised the robot measures about $25.5 \times 16.5 \times 22.2$ cm: inside the $30 \times 30 \times 30$ cm competition limit (R12) in every arm position, with about 4.5 cm margin in the longest direction.

3.2 Placement of the components

Component	Position	Reason
Motor driver board with the Pico	upper deck, back , next to the battery holder	short, thick battery wires to the board that takes the highest current; the Pico's USB connector stays reachable for programming
Battery holder (2 × 18650)	upper deck, back (kit position)	the on/off switch is easy to reach; the mass of the cells is at the back and balances the arm at the front
Servo driver (PCA9685 board)	lower deck, near the arm base	short servo cables to the arm
Pololu 5 V regulator	lower deck, next to the servo driver	short, low-resistance 5 V wires to the servo power terminal
WAGO 1, 2, 3 power connectors	lower deck	central star point of the power wiring (section 4.6)
Line sensor (Maker Line)	under the front of the robot, about 5–10 mm above the floor	the sensor sees the line before the wheels reach it, so the program has time to steer; its working distance is 4–40 mm
Ultrasonic sensor (HC-SR04)	front, about 4.5 cm above the floor (kit mount)	looks in the direction of travel and towards objects in front of the gripper
Arm	front of the upper deck	the gripper reaches the floor and objects in front of the robot; the robot drives up to an object nose first

All boards are fixed with **3M Command strips** (hook-and-loop type) instead of screws (principle P6). A board can be pulled off, checked or exchanged in seconds, and nothing has to be drilled into the kit's plates.

3.3 Drivetrain: why mecanum wheels

Task 3 asks the driver to pass obstacles **diagonally** and to park in garage cells that can be entered **only sideways**. A robot that can do this must be **holonomic**: it must be able to move in any direction in the plane without first turning. The usual drive types compare as follows:

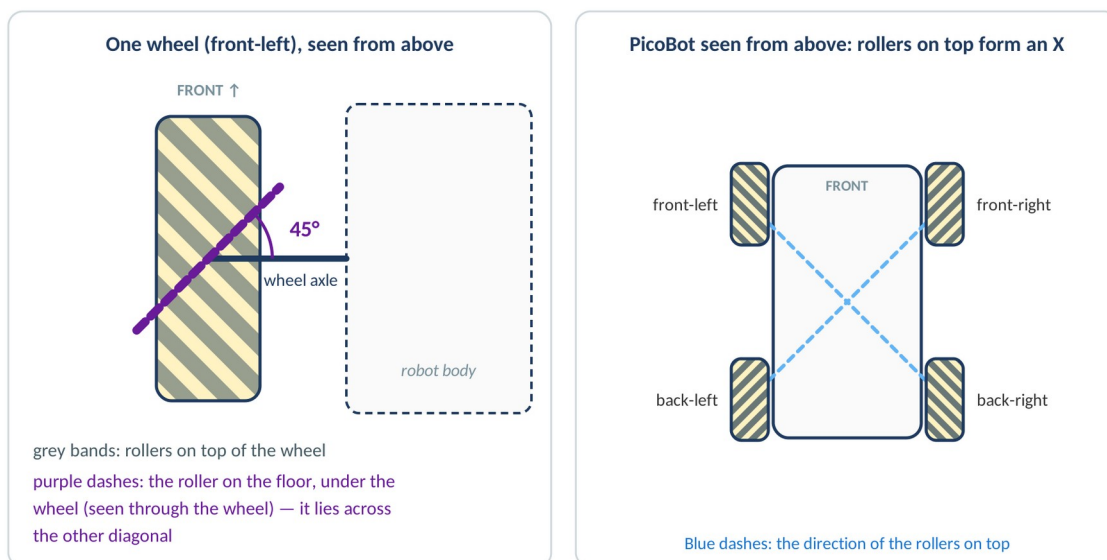
Drive type	Can move sideways?	Notes for PicoBot
Differential drive (2 driven wheels + castor)	no	simplest and cheapest; to move sideways it must turn, drive and turn back — fails R2
Car-like (Ackermann) steering	no	needs room to manoeuvre; parking in a “sideways-only” cell is impossible

Drive type	Can move sideways?	Notes for PicoBot
Omni wheels (3 wheels at 120°, or 4 at 90°)	yes	holonomic, but the motors must be mounted at angles to each other: does not fit a simple rectangular chassis
Mecanum wheels (4, in an “X” pattern)	yes	holonomic with all four wheels parallel , like a car: standard motors on a rectangular chassis; also shows the addition of force vectors very clearly

Mecanum wheels were therefore chosen: they meet R2 with the same simple motor mounting as an ordinary four-wheel car. The price is lower efficiency and some wheel slip in sideways and diagonal moves (section 3.6).

3.4 How a mecanum wheel works

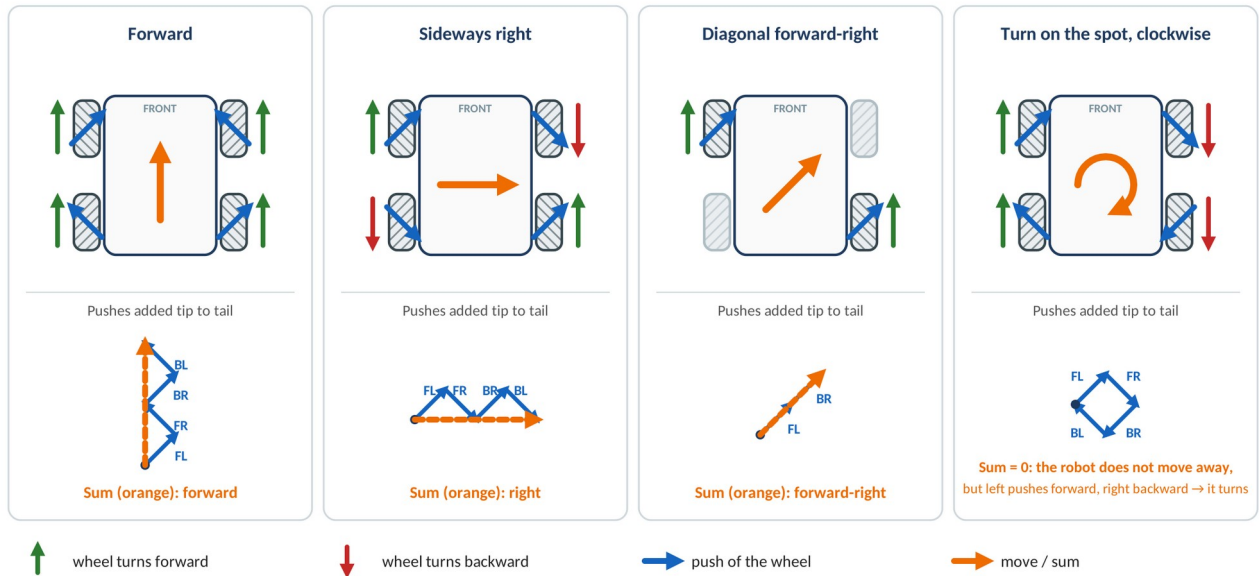
Each mecanum wheel carries a ring of free-turning **rollers** whose axes are set at **45°** to the wheel axle. When the wheel turns, the roller touching the floor cannot push along its own axis (it would just roll), so the floor pushes the wheel **at 45°**, across the roller. Each wheel therefore produces a diagonal push. Two “left-hand” and two “right-hand” wheels are mounted so that, **seen from above, the axes of the top rollers point towards the centre of the robot** (the “X” pattern).



Left: one mecanum wheel seen from above — the rollers on top and the roller on the floor lie on opposite diagonals. Right: the rollers on top of the four wheels form an X.

By choosing the direction of each wheel, the four diagonal pushes add up to any wanted movement:

Adding the pushes of the four mecanum wheels (PicoBot seen from above)



Adding the pushes of the four wheels for four moves. Green/red arrows: wheel turns forward/backward; blue: push of the floor on each wheel; orange: resulting movement.

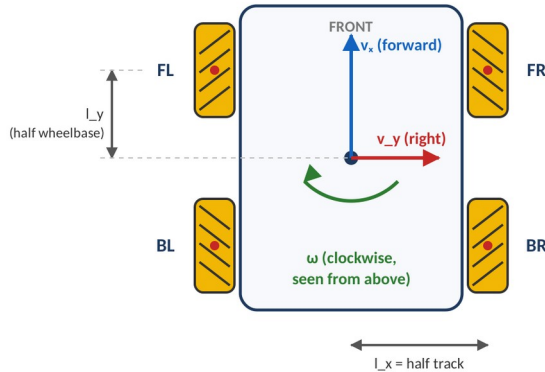
The control software (section 5.3) uses these wheel patterns. F = forward, B = backward, – = no drive:

Movement	Front-left	Back-left	Front-right	Back-right
Forward / backward	F / B	F / B	F / B	F / B
Sideways right	F	B	B	F
Sideways left	B	F	F	B
Diagonal forward-right	F	–	–	F
Diagonal forward-left	–	F	F	–
Diagonal backward-right	–	B	B	–
Diagonal backward-left	B	–	–	B
Turn on the spot clockwise	F	F	B	B
Turn on the spot counter-clockwise	B	B	F	F

In a diagonal move only two wheels are driven; the other two turn freely on their rollers. Their pushes would point across the wanted direction, so they are left without drive.

3.5 Kinematics

Mecanum geometry and sign conventions used in this document



Inverse kinematics (wheel surface speeds)

$$v_{FL} = v_x + v_y + (l_x + l_y) \cdot \omega$$

$$v_{BL} = v_x - v_y + (l_x + l_y) \cdot \omega$$

$$v_{FR} = v_x - v_y - (l_x + l_y) \cdot \omega$$

$$v_{BR} = v_x + v_y - (l_x + l_y) \cdot \omega$$

Wheel speed in rpm: $n = 60 \cdot v / (2\pi \cdot r)$, r = wheel radius

Firmware form (inputs f, s, r in $-100 \dots 100$ %):

$$FL = f + s + r$$

$$BL = f - s + r$$

$$FR = f - s - r$$

$$BR = f + s - r$$

If any $|value| > 100$, all four are scaled down by the same factor, so the direction of motion is kept.

Rollers at 45°; roller axes point to the centre (X pattern).

PicoBot mecanum geometry

Geometry and sign conventions: v_x forward, v_y to the right, ω clockwise seen from above; l_x = half the track width, l_y = half the wheelbase.

For a mecanum robot with 45° rollers in the X pattern, the speed that each wheel must have at its rim to give the robot the forward speed v_x , the sideways speed v_y and the turning rate ω is (inverse kinematics):

Wheel	Rim speed
Front-left	$v_{FL} = v_x + v_y + (l_x + l_y) \cdot \omega$
Back-left	$v_{BL} = v_x - v_y + (l_x + l_y) \cdot \omega$
Front-right	$v_{FR} = v_x - v_y - (l_x + l_y) \cdot \omega$
Back-right	$v_{BR} = v_x + v_y - (l_x + l_y) \cdot \omega$

Setting one of v_x , v_y , ω at a time gives exactly the table of section 3.4: for example $v_y > 0$ alone gives FL +, BL -, FR -, BR + (“sideways right”); $v_x = v_y$ gives FL and BR at double speed and BL, FR at zero (“diagonal forward-right”). The equations also show why turning on the spot is slower than driving straight: the wheel speed needed grows with the distance ($l_x + l_y$) of the wheels from the centre.

The joystick control program uses the same equations with percentage inputs: forward f , sideways s and rotation r , each from -100 to $+100$, give **FL = $f + s + r$** , **BL = $f - s + r$** , **FR = $f - s - r$** , **BR = $f + s - r$** . If any result is larger than 100 %, all four are divided by the same factor, so the **direction** of motion is kept and only the speed is reduced (section 5.3).

Geometry of PicoBot. Wheelbase 10.2 cm $\rightarrow l_y \approx 5.1$ cm. The overall width of 16.5 cm minus one wheel width of 3 cm gives a track of about 13.5 cm $\rightarrow l_x \approx 6.75$ cm. So **$l_x + l_y \approx 11.9$ cm**. Wheel diameter ≈ 6 cm $\rightarrow$ circumference ≈ 18.8 cm.

Speed (estimate). A TT gear motor with 1:48 gearbox turns at about 200 rpm without load at 6 V (section 3.6). With a 6 cm wheel this is $200 \times 18.8 \text{ cm} / 60 \text{ s} \approx \mathbf{0.63 \text{ m/s}}$ forward — an upper limit; with the robot’s weight, roller losses and a lower average motor voltage the real speed is lower. The default speed setting of the software (50 %) gives roughly half of this. The highest turning rate on the spot follows from $\omega = v / (l_x + l_y)$: $0.63 \text{ m/s} \div 0.119 \text{ m} \approx 5.3 \text{ rad/s} \approx \mathbf{300^\circ/\text{s}}$ (upper limit). Section 9 shows how to measure the real values.

3.6 Motors and practical limits of the drive

The kit uses four yellow “**TT**”-type **DC gear motors** with a plastic 1:48 gearbox, the most common motor of educational robot kits. The seller gives no data sheet; typical published data of this motor type are:

TT gear motor, 1:48 (typical published data)	Value
Rated voltage	3–6 V
No-load speed	about 90 rpm at 3 V, about 200 rpm at 6 V
No-load current	about 0.15 A
Stall torque	about 0.4 kg·cm at 3 V, about 0.8 kg·cm at 6 V
Stall current	about 1.5 A at 6 V

Why these motors are adequate. At 0.8 kg·cm and a wheel radius of 3 cm, one motor can push with about 0.27 kgf (≈ 2.6 N) at stall; four motors together ≈ 10 N. A robot with a mass of the order of 1 kg on a smooth floor needs only a small part of this to accelerate, so the motors are not the limit for the competition tasks. Their low price and universal availability (R13) and the fact that they are already part of the kit decided the choice.

Voltage note — an important design point. On PicoBot the motors are switched to the **battery voltage** (7.0–8.4 V) by the H-bridges, which is **above the 3–6 V rating** of TT motors. The speed command is a PWM duty cycle (section 4.3), so the **average** motor voltage is duty $\times$ battery voltage: the default 50 % gives about 3.7–4.2 V, well inside the rating. At 100 % the motors get the full 8.4 V of fresh cells, which makes them faster but also hotter and shortens the life of brushes and gears. Section 8 recommends a software limit of about 70 % (≈ 6 V average with full cells) for long runs at full speed.

Practical limits of the mecanum drive.

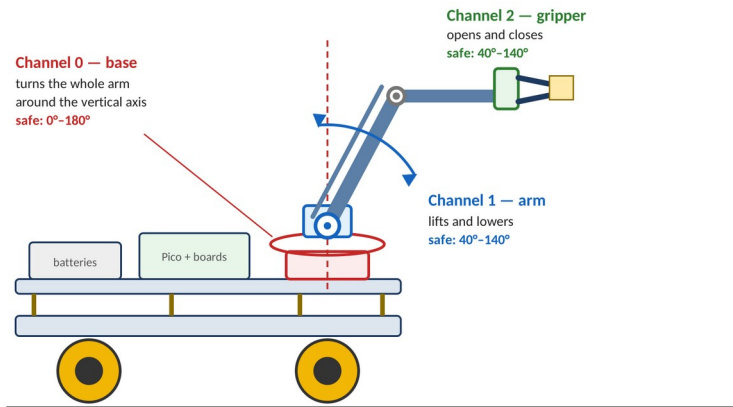
- **Slip and drift.** Each roller touches the floor only along a short line; in sideways and diagonal moves part of the motor work is lost in roller slip. Sideways moves are slower than forward moves, and a robot without wheel encoders drifts slightly. This is a property of every mecanum drive, not a fault.
- **Floor.** The rollers need a hard, flat floor. On carpet or on a surface with deep joints they sink or catch, and sideways motion becomes unreliable.
- **All four wheels must touch the floor.** If the chassis twists or one wheel loses contact, the vector sum is wrong and the robot turns. The rigid two-plate chassis and the even mass distribution (battery at the back, arm at the front) help here.
- **Unequal motors.** Cheap motors differ a few percent in speed. Over longer distances this makes the robot curve; the software compensates for this with calibration (measured times and speed factors), not with sensors.
- **Open loop.** There are no wheel encoders: distances and angles are produced by **time**, so they depend on battery voltage and floor. For the reference tasks this is acceptable because the line sensor and the ultrasonic sensor give the absolute references (line, marker, obstacle), and in the manual tasks the human driver closes the loop.

3.7 Robot arm

The arm of the kit has **two joints plus a gripper**, each moved by one micro servo:

Servo	Driver channel	Job	Safe range	Home
Base	0	turns the whole arm about a vertical axis (left/right)	0°–180°	90° (straight ahead)
Arm	1	lifts and lowers the arm	40°–140°	90°
Gripper	2	opens and closes the jaws	40°–140°	90°

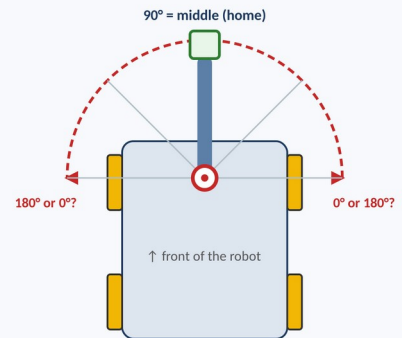
Side view



Home position: all three servos at 90°. Servo driver PCA9685 on I2C1 (GP2 SDA, GP3 SCL).

Top view

The base can point the arm anywhere in this half circle. Which side is 0° depends on how the servo horn is fitted.

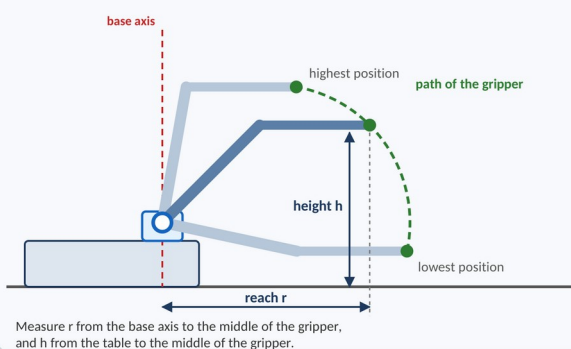


The PicoBot arm

The PicoBot arm: base (channel 0), arm (channel 1) and gripper (channel 2) with their safe angle ranges.

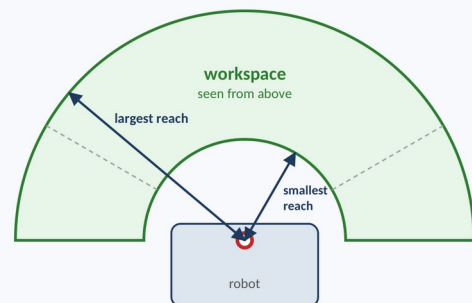
Why two joints are enough. On its own the arm has only 2 degrees of freedom (DOF) for positioning: turn and lift. But PicoBot is a **mobile manipulator**: the base adds three more DOF in the plane (x, y and rotation). Base and arm together can bring the gripper to any point in front of the robot at floor level — exactly what tasks 2 and 4 need (pick up a small object, put it down at a target). A 5- or 6-axis arm would add cost, mass at the front, servo current and programming complexity without being needed for these tasks. The small number of joints also makes the arm a clear example for teaching degrees of freedom and workspace.

Side view: the gripper moves on an arc



Measure r from the base axis to the middle of the gripper, and h from the table to the middle of the gripper.

Top view: a half ring



Base 0°-180° makes the half circle; the arm (lowest ↔ highest) makes the ring. Above each point of the ring, the gripper can be at only one height (2 DOF).

Workspace of the PicoBot arm (schematic)

Workspace of the arm (schematic): the gripper moves on an arc when the arm servo turns (side view) and covers a half ring when the base turns from 0° to 180° (top view).

Why the safe ranges. Below 40° or above 140° the arm servo would drive the arm into the chassis or against its mechanical stops; the gripper servo would press the jaws against each other or open them against the linkage. A servo pushing against a stop draws its stall current continuously, heats up and can strip its plastic gears — tests showed that over-tightening the gripper is the most likely way to damage a servo. The safe ranges, enforced in the software, are the design answer to this (sections 5.6 and 7.2).

Why the home position is 90°. Each servo is set to 90° by a program **before** its horn (the plastic lever on the shaft) is fitted, and the arm is assembled in its middle position. Then 90° is the centre of the mechanical range and the full safe range is available in both directions. The software moves all servos to 90° when it starts, so the arm always begins from a known, safe pose.

Servos. The kit is supplied with 9-gram micro servos of the **SG90** class (plastic gears); the metal-gear **MG90S** of the same size is the recommended replacement:

	SG90 (kit class)	MG90S (recommended replacement)
Supply	4.8–6 V (5 V typical)	4.8–6 V
Stall torque	about 1.8 kg·cm at 4.8 V	1.8 kg·cm at 4.8 V, 2.2 kg·cm at 6 V
Speed	about 0.1 s / 60° at 4.8 V	0.1 s / 60° at 4.8 V
Gears, mass	plastic, 9 g	metal, 13.4 g
Current (typical published)	about 10 mA idle, 100–250 mA moving, about 0.36 A stalled	similar or higher

Load capacity (estimate). A servo with 1.8 kg·cm can hold 1.8 kg at 1 cm from its shaft, or 0.18 kg at 10 cm. The arm servo must hold the arm's own weight as well as the object, so practical payloads are a few tens of grams — enough for the foam cubes (about 3 cm) and small 3D-printed objects of the reference tasks, and the reason why heavy or large objects are out of scope.

3.8 Mechanical robustness

Robustness and easy repair (R10) are achieved by:

- **Few moving parts:** four wheels and three servos; no belts, no gears outside the motor and servo housings.
- **Removable mounting** of all boards (P6), so a fault is repaired by swapping a board, not by rebuilding.
- **Pluggable wiring:** motor wires in screw terminals, power wires in WAGO lever connectors, signals on DuPont jumpers — no solder joints that can crack from vibration, except the header pins of the regulator.
- **Servo protection** by software limits (section 3.7) — the most frequent failure of small arms is a stripped servo gear.
- **Recommended spare parts:** one wheel, one motor, two or three servos (preferably metal-gear MG90S), DuPont wires.

4. Electronic design

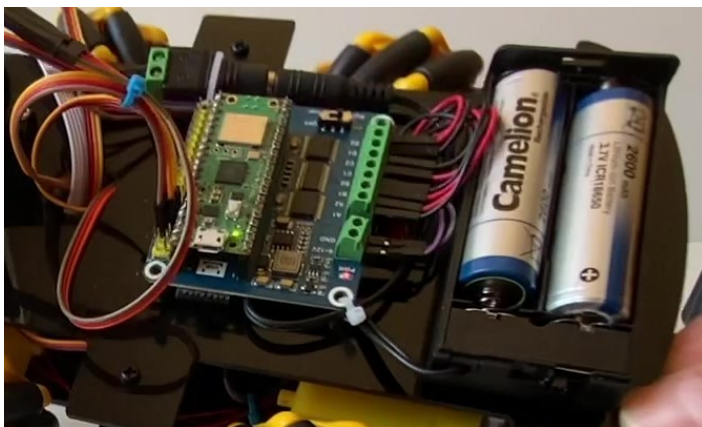
4.1 Architecture and interfaces

The electronics follow the principles of section 2.3: one controller, two intelligent drivers on two separate I2C buses, sensors read directly at 3.3 V, and three separate power branches from one battery.

Interface	From → to	Signal	Level / speed
I2C0	Pico GP20 (SDA), GP21 (SCL) → PCA9685 on the motor driver board	register writes (motor speed and direction)	3.3 V, 100 kHz, address 0x40
I2C1	Pico GP2 (SDA), GP3 (SCL) → PCA9685 servo driver board	register writes (servo pulse widths)	3.3 V, address 0x40
PWM, motor	motor-board PCA9685 → 2 × TB6612FNG	speed (duty cycle) and direction for 4 motors	3.3 V logic, 50 Hz
Motor outputs	TB6612FNG → 4 motors (terminals A–D)	switched battery voltage	0 V to battery voltage
PWM, servo	servo-driver PCA9685 → 3 servos	pulse 0.5–2.5 ms every 20 ms	50 Hz

Interface	From → to	Signal	Level / speed
Digital inputs	Maker Line D1–D5 → GP8, GP9, GP13, GP14, GP15	1 = sensor sees the line	3.3 V
Trigger / echo	GP27 → HC-SR04 Trig; HC-SR04 Echo → GP26	10 μ s trigger pulse; echo pulse width = time of flight	3.3 V
UART0 (optional)	GP0 (TX) → JDY-31 RXD; JDY-31 TXD → GP1 (RX)	serial data	3.3 V, 9600 baud default
Wi-Fi	Pico's CYW43439 chip ↔ phone / PC	802.11n, 2.4 GHz, robot is the access point	—
USB	computer → Pico	programming, file transfer, REPL; 5 V supply for the Pico only	USB 1.1

4.2 Controller: Raspberry Pi Pico 2 W



The Pico 2 W plugged into the Waveshare motor driver board on PicoBot.

	Pico 2 W (standard on PicoBot)	Pico W (also supported)
Microcontroller	RP2350	RP2040
Cores	2 × Arm Cortex-M33 or 2 × RISC-V Hazard3, up to 150 MHz	2 × Arm Cortex-M0+, 133 MHz
SRAM / flash	520 KB / 4 MB	264 KB / 2 MB
Wireless	Infineon CYW43439: Wi-Fi 802.11n 2.4 GHz and Bluetooth 5.2, on-board antenna; connected to the microcontroller by SPI	same
GPIO	26 user pins, 3 of them analog inputs (GP26–GP28)	same pinout
Power input VSYS	1.8–5.5 V; on-board RT6154 buck-boost converter makes 3.3 V	same principle
3V3 (OUT)	keep the external load below 300 mA (data sheet recommendation)	same
USB	micro-USB; VBUS → VSYS through a Schottky diode	same

Why the Pico. The Raspberry Pi Pico W family (R1) is technically a very good choice for an educational robot:

- **Wi-Fi on the board** — the robot can create its own access point and serve a control web page (R6) without any extra module, router or app.

- **MicroPython** is officially supported by Raspberry Pi; programs are plain Python files that students can read and change on the robot with the free Thonny editor, with an interactive shell (REPL) for experiments (R1, R7).
- **Plug-in format:** the 40-pin board plugs straight into the motor driver base board — no wiring for the main connections (R9).
- **Low cost, low power, robust start-up:** a microcontroller starts in about a second and cannot corrupt a file system when the battery is switched off, unlike a Linux single-board computer.
- **Enough reserves (R14):** two cores, 4 MB flash, 13 free GPIO pins, a free analog input and the programmable I/O (PIO) blocks for later extensions.

Alternative	Why it was not chosen
Arduino Uno (the kit's own board)	no Wi-Fi; programmed in C++, not Python; 5 V logic
ESP32 boards	have Wi-Fi and run MicroPython, but the many board variants differ in pinout and USB chip, and there is no plug-in motor base board of the same quality
Raspberry Pi (Linux)	higher current and cost, long boot time, risk of SD-card corruption when power is cut

Pico 2 W and Pico W. The two boards have the same size and pinout and run the same MicroPython programs; only the MicroPython firmware file differs (RPI_PIC02_W-... .uf2 or RPI_PICO_W-... .uf2). Schools can use whichever they can buy.

Pins used inside the board. GP23, GP24, GP25 and GP29 connect the wireless chip (power-on, SPI data, chip select, clock) and are not available on the header. The green LED belongs to the wireless chip and is addressed as Pin ("LED").

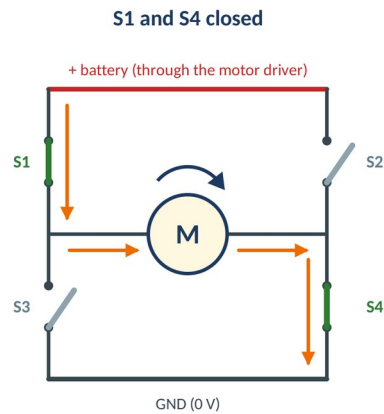
Logic level. The GPIO pins work at **3.3 V**. The data sheet allows on the analog pins at most 0.3 V above the I/O supply; the older RP2040 of the Pico W is not 5 V tolerant at all. PicoBot therefore never connects a 5 V signal to a Pico pin (principle P2) — this keeps the design valid for both boards.

4.3 Motor driver: Waveshare Pico-Motor-Driver

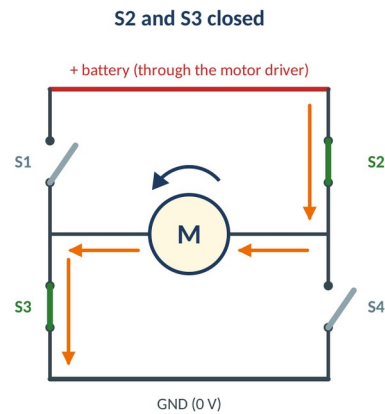
The **Waveshare Pico-Motor-Driver** (product 19764) is the base board of PicoBot. The Pico plugs into its header. It carries three functions:

Function	Component	Data
PWM generator	PCA9685 , on I2C0 (GP20 SDA, GP21 SCL), address 0x40 (5 address jumpers: 32 addresses possible)	16 channels, 12-bit
Power stages	2 × TB6612FNG dual H-bridge = 4 motor channels, terminals A–D	1.2 A average, 3.2 A peak per channel; motor supply up to 13.5 V
Supply	5 V regulator, up to 3 A, feeds the Pico (VSYN)	input VIN 6–12 V
Board	logic level 3.3 V; size 65 × 56 mm, 3 mm mounting holes	

How a DC motor is driven: the H-bridge. A DC motor turns one way or the other depending on the direction of the current. An H-bridge is four electronic switches around the motor: closing one diagonal pair lets current flow one way, closing the other pair reverses it.



Current flows through the motor one way
library: "forward" — direction channels IN1 = 0, IN2 = 1



Current flows the other way: the motor turns the other way
library: "backward" — IN1 = 1, IN2 = 0

Orange arrows = path of the current. All switches open = motor off. S1 + S3 (or S2 + S4) together would be a short circuit — the driver chip never does this.

Principle of the H-bridge: the closed diagonal pair of switches decides the direction of the current and therefore of the motor.

The TB6612FNG builds these switches from **MOSFET transistors** with low on-resistance. Its control inputs per channel are IN1, IN2 (direction) and PWM (speed):

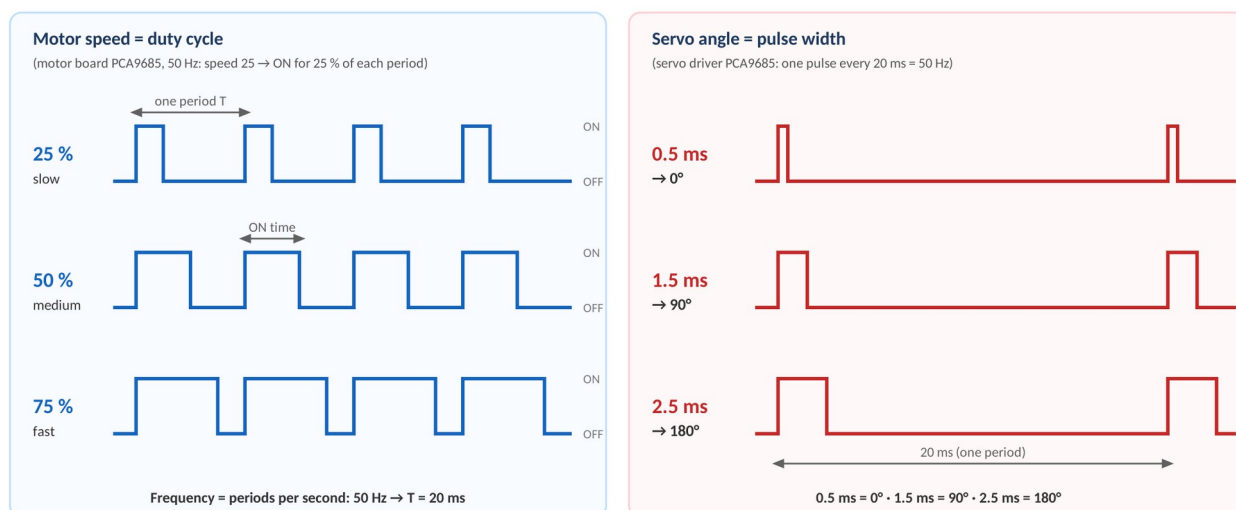
IN1	IN2	PWM	Output
H	L	H	turns one way (CW)
L	H	H	turns the other way (CCW)
H	L	L	short brake : both motor terminals connected together, the motor stops quickly
L	H	L	short brake
H	H	any	short brake
L	L	H	stop : outputs switched off, the motor coasts

(The STBY input must be high to enable the chip; the PicoBot software has no standby control, so the board enables the drivers itself.)

How the speed is set: PWM. The motor driver does not make a lower voltage for a slower speed. It switches the full battery voltage on and off many times per second; the **duty cycle** — the share of time the switch is on — sets the average voltage and therefore the speed.

PWM — switching on and off very fast

The period stays the same. The duty cycle — the share of each period that the signal is ON — sets how much power arrives on average.



PWM: for the motors the duty cycle sets the average power (left); for the servos the width of the pulse sets the angle (right).

How the Pico commands a motor. The Pico does not produce these signals itself. It writes register values over I2C into the PCA9685, which then generates all PWM and direction signals in hardware, continuously, even while the Pico is busy with Wi-Fi. Each motor uses **three PCA9685 channels**:

Terminal	Motor	Name in the software	PWM (speed)	IN1, IN2 (direction)
A1 (+, red) / A2 (-, black)	M1 front-left	"LeftFront"	0	1, 2
B1 / B2	M2 back-left	"LeftBack"	3	4, 5
C1 / C2	M3 front-right	"RightFront"	6	7, 8
D1 / D2	M4 back-right	"RightBack"	9	10, 11

A direction channel is set fully on (4095 of 4095) or fully off (0) and so acts as a logic level; forward is IN1 = 0, IN2 = 1, backward IN1 = 1, IN2 = 0. The speed 0–100 % is converted to 0–4095 on the PWM channel. When the installation test shows a wheel turning backwards, the two motor wires are swapped at the terminal — a deliberate design choice that keeps the software identical on every robot.

Stopping uses the truth table above: setting only the PWM channel to 0 while the direction stays set gives a **short brake**; setting all three channels to 0 switches the outputs **off** and lets the motor coast. The library's `StopAllMotors()` does the latter for all four motors.

PWM frequency. The PCA9685 has one frequency for all 16 channels of a chip. The motor library sets the motor-board chip to **50 Hz** (period 20 ms) — the same value as for servos, inherited from the manufacturer's example code. It works, but such a low frequency makes the motors run in audible steps at low speeds; section 8 discusses raising it.

Why this board.

- **No wiring of logic signals** — the Pico plugs in, the motors go to screw terminals (R9).
- **Only two Pico pins for four motors:** 12 control signals are produced by the PCA9685 from one I2C bus. This leaves the Pico's pins free for sensors and extensions (R14).

- **The input range 6–12 V matches a 2-cell Li-ion pack** (6.0–8.4 V), and the built-in 3 A regulator powers the Pico from the same battery (R8).
- **MOSFET H-bridges** waste little power.

Alternative	Comparison
L298N modules (common in kits)	bipolar transistors lose about 2 V in the bridge — a quarter of the battery voltage; need a large heat sink, 5 V logic, and 3 Pico pins per motor (12 pins for 4 motors)
DRV8833 modules	efficient, but limited to 10.8 V and need two PWM pins per motor from the Pico (8 pins), plus separate wiring
The kit's Arduino expansion board	made for the Arduino Uno, 5 V logic

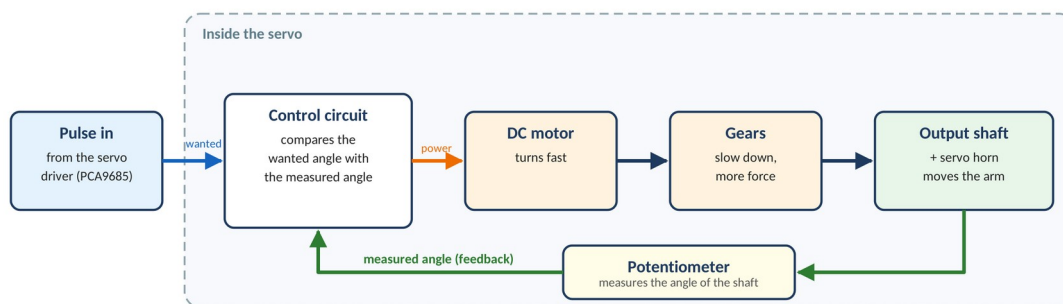
4.4 Servo driver: PCA9685 16-channel board

The arm servos are driven by a **separate PCA9685 board** on the second I2C bus (I2C1: GP2 SDA, GP3 SCL, address 0x40).

PCA9685 (NXP)	Value
Channels, resolution	16 PWM outputs, 12 bit (4096 steps per period)
Frequency	24 Hz–1526 Hz, one frequency for all channels; internal 25 MHz oscillator
I2C	up to 1 MHz (Fast-mode Plus); 6 address pins → up to 62 devices on one bus; “All Call” address 0x70
Supply (logic)	2.3–5.5 V; inputs 5.5 V tolerant
Outputs	totem pole, 25 mA sink / 10 mA source at 5 V; active-low output-enable pin

On the board the chip's logic is powered at the side connector (**VCC = 3.3 V**), while the servos get their power from a separate screw terminal (**V+ = 5 V** from the Pololu regulator, section 4.6), which is distributed to the 16 three-pin servo headers.

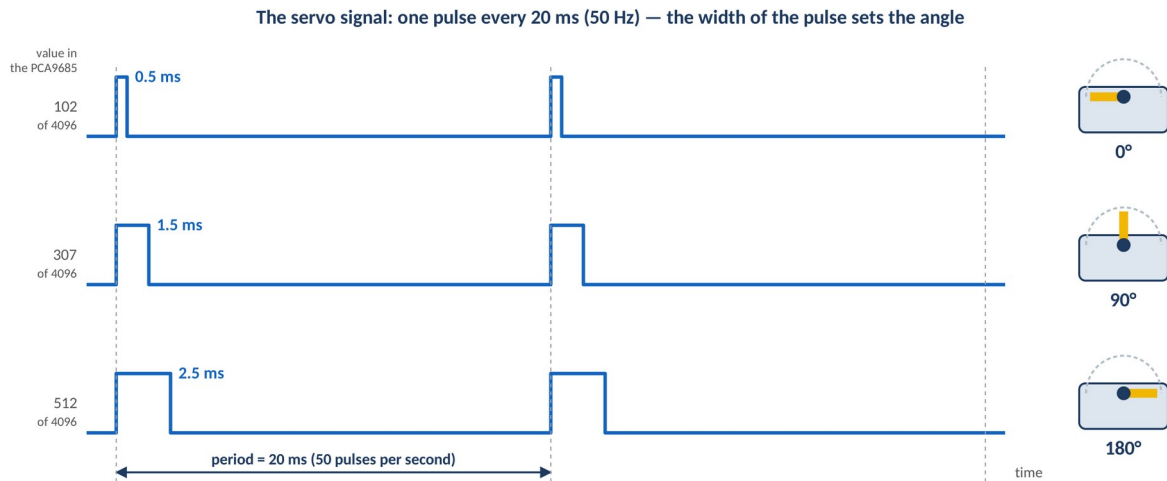
How a servo is controlled. A hobby servo contains a small DC motor, a gearbox, a potentiometer on the output shaft and a control circuit. It receives a pulse every 20 ms; the **width of the pulse** is the wanted angle. The control circuit compares it with the potentiometer and drives the motor until both agree.



The control circuit drives the motor until the measured angle equals the wanted angle — then the motor stops.
If a force pushes the arm away, the circuit notices it and pushes back. This is called closed-loop control.

Inside a hobby servo

Inside a hobby servo: motor, gearbox, potentiometer and control circuit form a closed position loop.



Between the pulses the signal is 0 V. The servo only looks at how long each pulse is. (Drawn to scale.)

Servo pulse width

The servo signal on PicoBot: one pulse every 20 ms; 0.5 ms = 0°, 1.5 ms = 90°, 2.5 ms = 180°.

At 50 Hz one period of 20 ms is divided into 4096 steps of 4.88 μ s. The library converts an angle to a step count: 0° → 102 steps (0.50 ms), 90° → 307 steps (1.50 ms), 180° → 512 steps (2.50 ms). The resolution is $(512 - 102) / 180 \approx 2.3$ steps per degree, i.e. **about 0.44°** — finer than the dead band of a micro servo, so the driver never limits the accuracy of the arm.

Why a separate servo driver, and why not the Pico's own PWM.

- **Different PWM frequency.** Servos need 50 Hz; the PCA9685 sets one frequency for the whole chip. A separate chip for the servos leaves the motor chip free to use any frequency that suits the motors (section 8).
- **Hardware timing.** The servo pulses are produced by the PCA9685 continuously and precisely, whatever the Pico is doing (Wi-Fi, web server, garbage collection). Pulses generated in software would jitter, and a jittering pulse makes a servo twitch.
- **Separate power path.** The board's V+ terminal lets the servo current flow from the 5 V regulator to the servos without passing through the Pico or the motor board (principle P1).
- **Reserve.** 13 of 16 channels are free for more servos, lights or other PWM devices (R14).
- **Why its logic runs at 3.3 V.** On the common PCA9685 boards the I2C pull-up resistors are connected to VCC. With VCC = 3.3 V the SDA and SCL lines never rise above 3.3 V at the Pico pins (principle P2).

Servo connection. Brown → GND (black pin), red → V+ (red pin), orange → signal (yellow pin). Channels: 0 = base, 1 = arm, 2 = gripper.

4.5 Battery

Cells	2 × 18650 Li-ion , 2600–3600 mAh, standard size without protection circuit, from a known brand
Connection	in series in the kit's holder, which has an on/off switch
Voltage	7.4 V nominal; 8.4 V full (4.2 V per cell); recharge at 7.0 V (3.5 V per cell); absolute minimum about 6.0 V (3.0 V per cell)
Energy (estimate)	2 × 3.7 V × 2.6–3.6 Ah ≈ 19–27 Wh
Charging	external 4- or 6-bay 18650 charger (mains-powered preferred); 2–3 labelled pairs per robot for class use

Why two 18650 cells in series.

- **The voltage window fits the electronics.** 6.0–8.4 V is inside the 6–12 V input of the motor board, and always more than 5 V, so both 5 V step-down regulators can work over the whole discharge.
- **High current.** 18650 cells deliver several amperes without a large voltage drop — needed when four motors start together or stall.
- **Energy and availability.** One cell stores about 10 Wh, the format is sold everywhere, chargers are cheap, and the kit's holder already takes two cells (R8, R13).

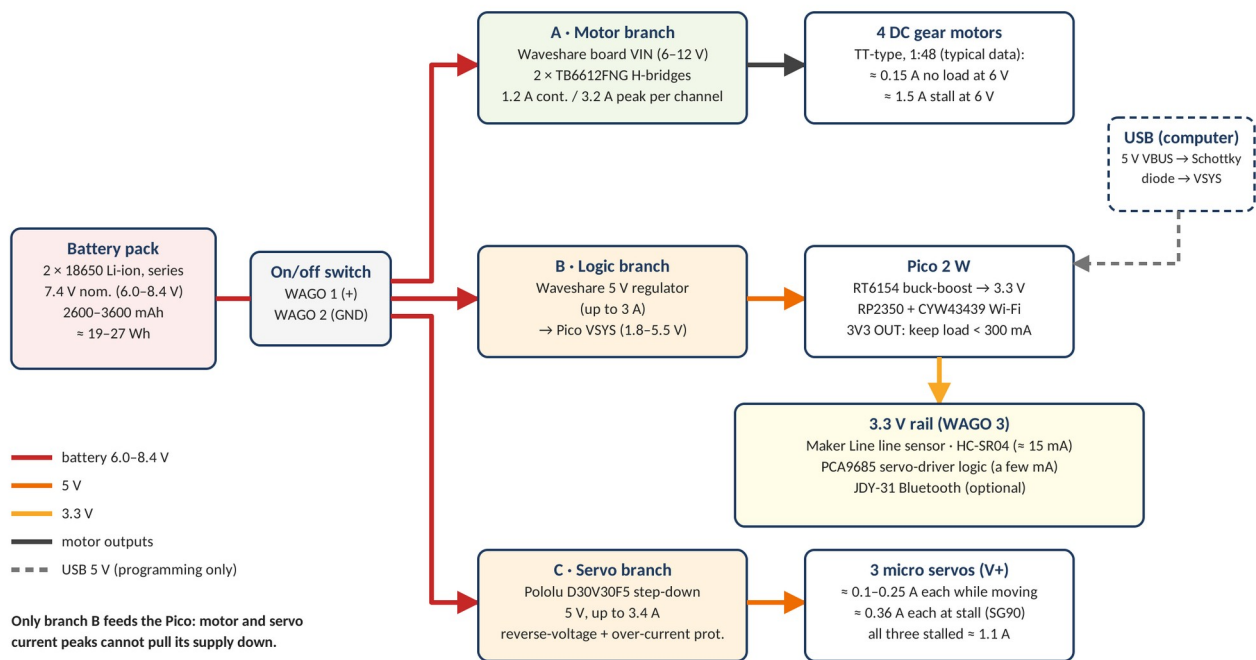
Alternative	Comparison
6 × AA NiMH (7.2 V)	similar voltage, but less energy (about 14 Wh), heavier, and it would need a different, larger holder
9 V block	far too little current and energy
LiPo pack (2S)	similar voltage, but needs a balance charger and more care (soft pouch can be punctured)

Why unprotected cells, and how the risk is handled. Cells with a built-in protection circuit are a few millimetres longer and do not fit the kit's holder. Without protection, the design relies on operating rules: recharge at **7.0 V** (well above the 6.0 V limit, which also gives many more charge cycles), charge each cell in its own charger bay, keep the two cells of a robot together as a labelled pair, switch the holder off during wiring, and store cells out of the robot. The optional **2-wire voltmeter** (2.5–40 V), connected after the switch, shows the pack voltage while the robot is on.

4.6 Power distribution

PicoBot power tree and current budget

Three independent branches from one battery pack. Current values are manufacturers' data or typical published figures, not measurements on PicoBot.



PicoBot power tree

PicoBot power tree: three branches from one battery, with typical currents.

From the switch, the battery feeds **WAGO 1** (battery +); **WAGO 2** is battery – and the **common ground** of the whole robot. Three branches leave WAGO 1:

Branch A — motors. Battery → motor board VIN → TB6612FNG → motors. The H-bridges switch the battery voltage directly; there is no regulator in the motor path, so no regulator can limit the motor current or overheat.

Branch B — logic. Battery → motor board's **5 V regulator** (up to 3 A) → Pico **VSYS** (pin 39) → the Pico's own RT6154 converter → **3.3 V**. The Pico's **3V3 (OUT)** pin (pin 36), passed through on the motor board's header, feeds **WAGO 3**, the 3.3 V rail for the Maker Line, the HC-SR04, the servo driver logic and the JDY-31.

Branch C — servos. Battery → **Pololu D30V30F5** → 5 V → servo driver V+ → servos.

Why three branches (principle P1). The motors and servos draw large, short current peaks — at start-up, when they change direction, and when they stall. If the Pico shared their supply, each peak would pull its voltage down and could reset the microcontroller in the middle of a movement; the robot would stop responding and the web page would disappear. With separate branches, the Pico's supply sits behind its own regulator and only sees the battery voltage dropping slowly — which the 5 V regulator easily absorbs.

The Pololu D30V30F5 servo regulator.

Pololu D30V30F5	Value
Type	synchronous step-down (buck) switching regulator, fixed 5 V output ($\pm 4\%$)
Input	up to 45 V (minimum about 5 V plus the drop-out voltage at the load current)
Output current	up to 3.4 A (typical, depends on input voltage and cooling)
Efficiency	80–95 %
Protection	reverse-voltage protection up to 40 V, over-current and short-circuit protection, over-temperature shut-off, under-voltage lock-out
Extras	enable input (EN), power-good output (PG); switching frequency about 700 kHz
Size	17.8 × 20.3 mm

Why a switching regulator. A linear regulator (e.g. a 7805) turns the voltage difference into heat: at 1 A and 8.4 V input it would have to dissipate $(8.4 - 5) \times 1 = 3.4$ W — too much for a small robot without a heat sink. The D30V30F5 converts the same power at 80–95 % efficiency and stays cool.

Why 3.4 A. Three SG90-class servos draw about 0.36 A each when stalled (≈ 1.1 A together) and up to 0.25 A each while moving; metal-gear servos and simultaneous starts draw more. 3.4 A gives a large margin, so the servo voltage does not sag when all three servos start at once. (The smaller Pololu **D24V25F5**, 5 V 2.5 A, used in the first PicoBot prototypes, also works but leaves less headroom.)

Why its protections matter in a school. Reverse-voltage protection survives the most common wiring mistake (+ and – swapped); short-circuit and over-temperature protection survive a short on the servo wiring.

The 3.3 V rail. The Pico's 3V3 output should carry at most about 300 mA of external load. The loads on PicoBot are small: HC-SR04 about 15 mA, PCA9685 logic a few mA, the Maker Line's infrared emitters and the optional JDY-31 — together well below this limit (**estimate**; the Maker Line data sheet gives no current figure). Only pin 36 (3V3 OUT) may be used for this rail; pins 39 (VSYS) and 40 (VBUS) carry about 5 V.

Common ground. All GND wires meet at **WAGO 2** (a star point). Every signal — I2C, sensor outputs, UART — is measured against ground, so all parts must share it; the star point keeps the large motor and servo return currents on their own wires to the battery instead of flowing through signal ground wires.

USB and battery together. When the Pico is connected to a computer, USB VBUS feeds VSYS through the Pico's Schottky diode. The diode stops the robot's 5 V from flowing back into the computer, so USB and battery may be connected at the same time — the normal way to program and test the robot. USB alone powers only the Pico: motors, servos and sensors need the battery switched on.

WAGO lever connectors. The WAGO 221 series (5-way and 2-way, 0.2–4 mm², rated 32 A) connects several wires without soldering or screws: the lever is opened, the stripped wire inserted, the lever closed. They are far more reliable than a breadboard and much easier for beginners than soldering (P5).

Current budget and running time (estimate).

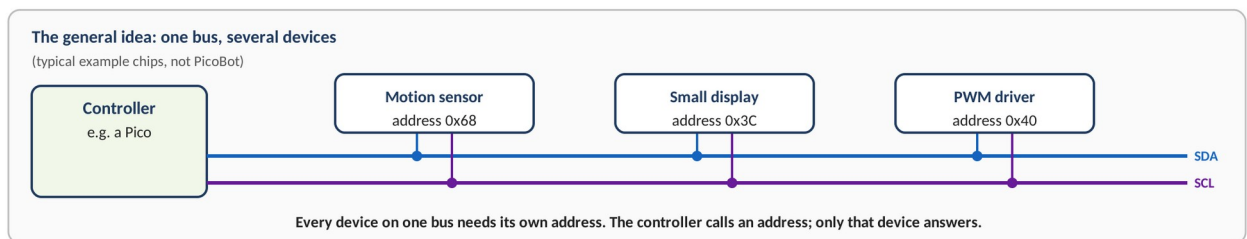
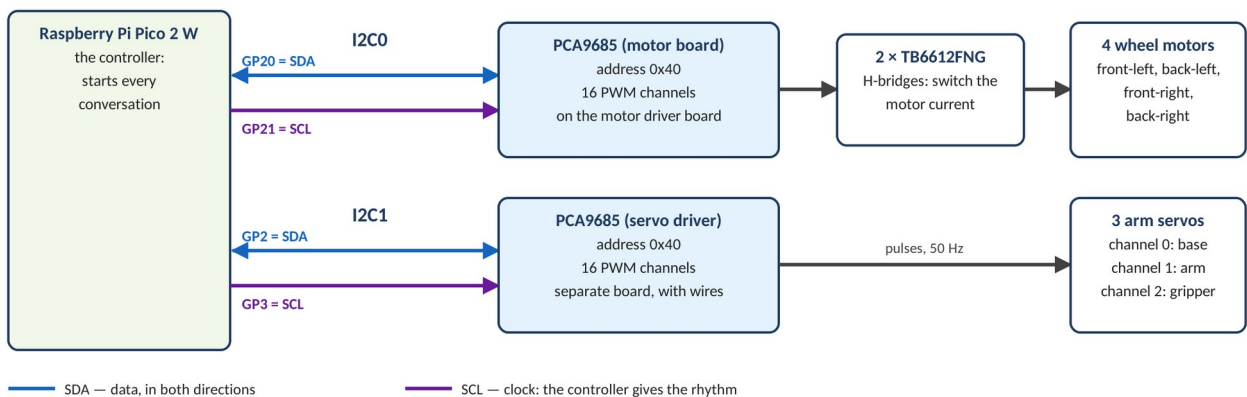
Consumer	Typical while driving	Peak / worst case
4 motors	about 0.2–0.4 A each under light load → 0.8–1.6 A	about 1.5 A each at stall (6 V data)
Pico 2 W with Wi-Fi access point	about 0.1 A	—
Sensors and driver logic (3.3 V rail)	below 0.1 A	—
3 servos	holding: tens of mA	3 × 0.36 A stalled (SG90 class)
Total	about 1–2 A	several amperes for short moments

With a 2600–3600 mAh pair and the 7.0 V recharge limit (assumed: about 70 % of the rated capacity usable), the robot can drive continuously for roughly **1–2.5 hours**, and much longer in a lesson with pauses. Section 9 shows how to measure it.

4.7 Communication buses

PicoBot has two I2C buses

Each bus = two signal wires (SDA data, SCL clock) + a common GND wire. Both driver chips have the address 0x40 — they are on different buses, so there is no conflict.



PicoBot has two I2C buses. Both PCA9685 chips have the address 0x40; on separate buses there is no conflict.

I2C (Inter-Integrated Circuit) is a two-wire bus: SDA carries the data in both directions, SCL the clock given by the controller. Every device on a bus has an address; the controller calls an address and only that device answers. On PicoBot:

Bus and Pico pins	Device and address	Wiring
I2C0: GP20 (SDA), GP21 (SCL)	PCA9685 on the motor driver board, address 0x40	tracks on the board — no wires
I2C1: GP2 (SDA), GP3 (SCL)	PCA9685 servo driver board, address 0x40	four jumper wires (VCC, GND, SDA, SCL)

A bus scan shows 0x40 and 0x70 on each bus: 0x70 is the PCA9685's “All Call” address, not a second device.

Why two buses instead of one.

1. **Same address.** Both PCA9685 chips have the default address 0x40. On one bus, one of them would have to be re-addressed — by changing the motor board's address jumpers or by soldering an address pad on the servo board. Two buses need no change at all (P5) and every robot is identical.
2. **Fault isolation.** I2C1 runs over jumper wires, which are the most likely place for a loose contact. A fault on I2C1 can block that bus; on a shared bus it would also block the motors — the robot could not even be stopped. With separate buses, the motors stay controllable when the arm bus fails (R10, R11).
3. **Independent software.** The motor driver and the arm driver each own one bus object; the drive loop and the arm loop never wait for each other.

I2C timing (estimate). The motor library writes each PCA9685 register in a separate I2C transaction (about 3 bytes $\approx$ 0.3 ms at 100 kHz). One `TurnMotor()` call writes 12 registers $\approx$ 3.6 ms, so a command for all four wheels takes about 15 ms. This is fast enough for the reference tasks; the joystick program writes the 4 registers of a channel in one burst, which halves the bus traffic (section 8).

UART (optional Bluetooth). The JDY-31 module (Bluetooth 3.0, serial port profile, 9600 baud by default) is connected to UART0: Pico TX (GP0) $\rightarrow$ module RXD, module TXD $\rightarrow$ Pico RX (GP1) — transmit always goes to receive. It is powered from the 3.3 V rail. The current programs control the robot over Wi-Fi and do not use the module. Note: the JDY-31 breakout boards are specified for 3.6–5 V on VCC; before Bluetooth is used in a program, its operation at 3.3 V should be checked.

Wi-Fi. The CYW43439 chip runs in **access-point mode**: the robot creates its own network, and the phone or PC connects to it directly; the robot's web page is at `http://192.168.4.1/`. No school network, router or internet access is needed — important because school networks often block unknown devices (R6). The programs set the regulatory country (for example `rp2.country('BG')` — change it to your own country code) so that the radio uses the right channels. Every program has a default network name (for example `picobot-web`); when several robots run the same program in one room, each robot needs its own network name, otherwise users connect to the wrong robot.

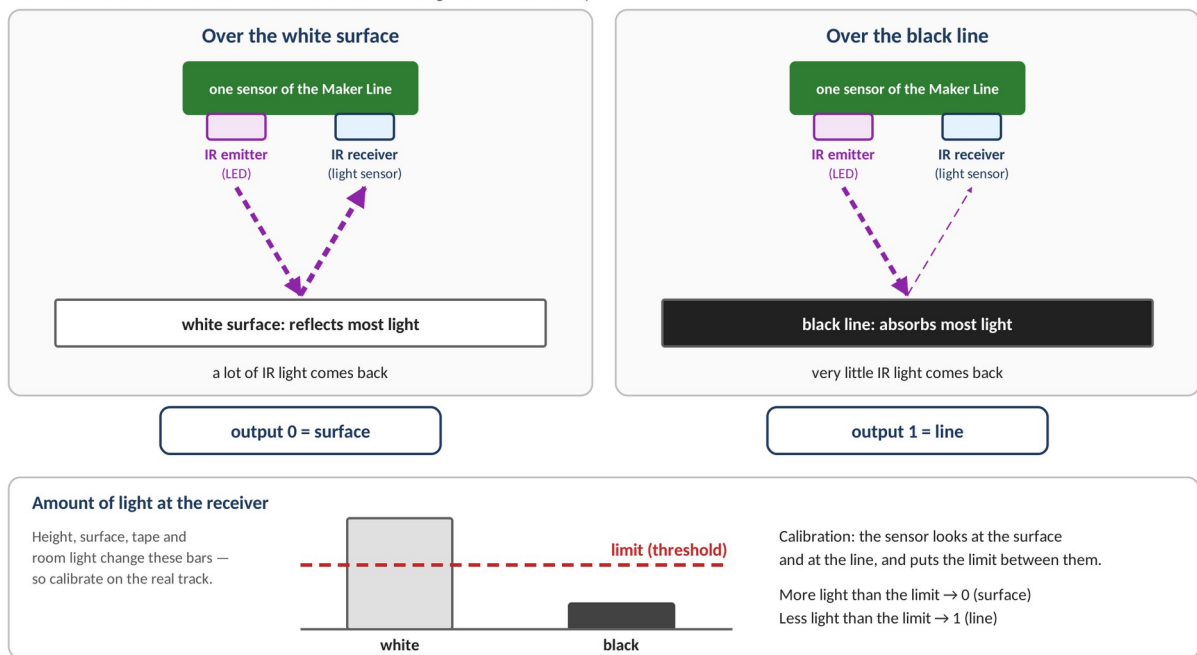
4.8 Sensors

4.8.1 Line sensor: Cytron Maker Line

Cytron Maker Line	Value
Supply	3.0–5.5 V, reverse-polarity protected (on PicoBot: 3.3 V)
Sensors	5 infrared emitter/receiver pairs in a row
Outputs	5 digital outputs D1–D5 (high = line detected) and one analog output (0–VCC = position of the line)
Line width	13–30 mm recommended
Sensing distance	4–40 mm
Refresh rate	200 Hz
Calibration	button: the sensor is moved across the line, the thresholds are stored in its EEPROM
Line colour	slide switch: dark line on light surface, or light line on dark surface

Infrared reflection: how the line sensor sees the line

Each of the 5 sensors has an IR emitter and an IR receiver. Infrared light is invisible to our eyes.



How the line sensor sees the line: a white surface reflects most infrared light, a black line absorbs it.

How it works. Each of the five sensors sends infrared light to the floor and measures how much comes back. After calibration the sensor's own microcontroller compares each value with a threshold and sets the digital output: 1 = line, 0 = surface. On PicoBot the five outputs go to **GP8, GP9, GP13, GP14, GP15** (D1 = right ... D5 = left), read as inputs with internal pull-up.

Why this sensor (replacing the kit's 3-channel sensor).

- **Five channels** give the position of the line in five steps (−2 ... +2) instead of three. The line-following program uses this for a weighted position and seven steering levels — smoother following and earlier correction (R4).
- **Marker detection.** A line across the track (the lap and stop markers of tasks 1 and 2) makes **all five** sensors see black at the same moment — a clear, unambiguous event that a 3-channel sensor with a narrower span detects less reliably.
- **One-button calibration** instead of one trimmer per channel: students calibrate the sensor for their track in seconds, and the setting survives switching off.
- **Works at 3.3 V**, so its outputs are safe for the Pico (P2).
- **Digital outputs** are directly readable: a list of five 0/1 values shows exactly what each sensor sees, which is ideal for teaching and debugging. The analog position output is a reserve: it could be connected to the free analog input GP28 (section 8).

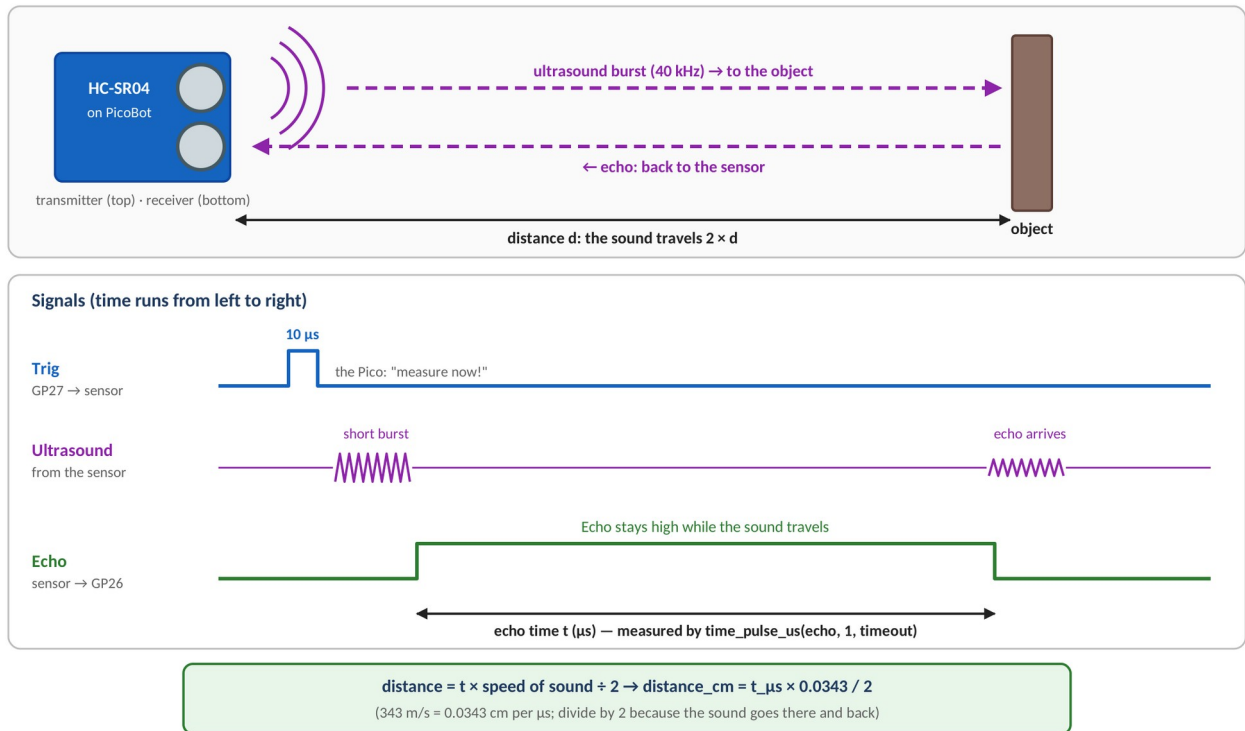
The sensor sits under the front of the robot, about 5–10 mm above the floor — inside its 4–40 mm range, and close enough that the room light has little effect.

4.8.2 Ultrasonic distance sensor: HC-SR04

HC-SR04 (classic data sheet)	Value
Supply	5 V, about 15 mA (on PicoBot: 3.3 V, see below)
Frequency	40 kHz
Range	2–400 cm; resolution about 3 mm
Beam	measuring angle about 15°
Trigger	pulse of at least 10 μs

HC-SR04 (classic data sheet)	Value
Echo	high pulse whose length is the travel time of the sound
Measurement cycle	at least 60 ms between triggers

Ultrasonic distance sensor HC-SR04: echo time → distance

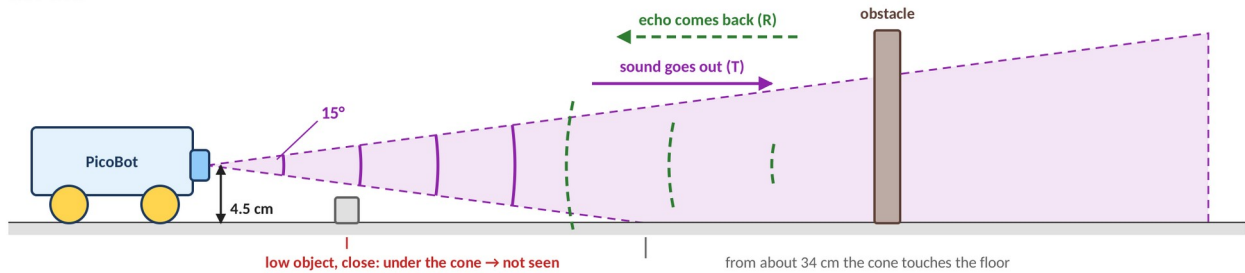


Echo timing: the Pico sends a 10 μs trigger pulse; the Echo output stays high while the sound travels to the object and back.

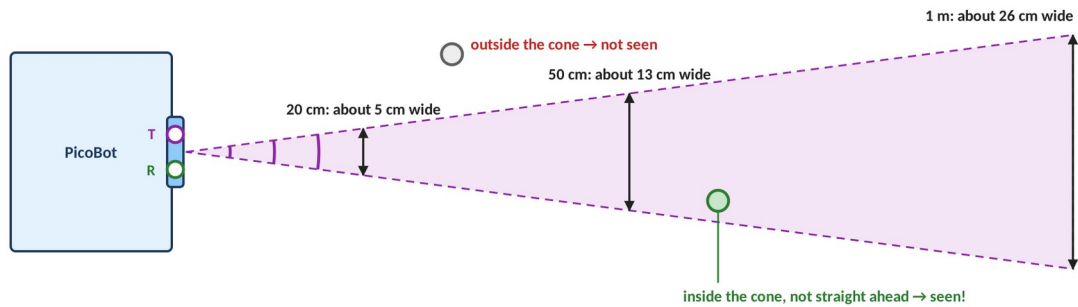
How it works. After the trigger pulse the module sends a short burst of 40 kHz sound and sets Echo high; when the echo returns, Echo goes low. The pulse length t is the time for the way there **and back**, so distance = $t \times \text{speed of sound} / 2$. With 343 m/s = 0.0343 cm/μs: **distance_cm = $t_{\mu\text{s}} \times 0.0343 / 2$** ($\approx t_{\mu\text{s}} / 58$).

The sound of the HC-SR04 spreads out like a cone

Side view



Top view



The sensor reports the distance to the nearest object anywhere inside the cone — not only straight ahead.
Width of the cone = distance ÷ 4 (data sheet: measuring angle 15°). Obstacles for PicoBot: at least 10 cm tall.

The sound spreads as a cone of about 15°: the sensor reports the nearest object anywhere inside the cone.

Why ultrasonic. It measures distance to objects of **any colour** — also black or transparent objects that confuse infrared sensors — over a long range (up to 4 m), costs very little and is part of the kit. Its principle (time of flight of sound) is easy to understand and to calculate. Its limits are known and manageable: soft objects (foam, cloth) absorb the sound, surfaces at an angle reflect it away, thin objects reflect too little, and several robots close together can hear each other's pulses. Obstacles for PicoBot should therefore be hard, have a flat front and be at least as tall as the sensor (about 10 cm).

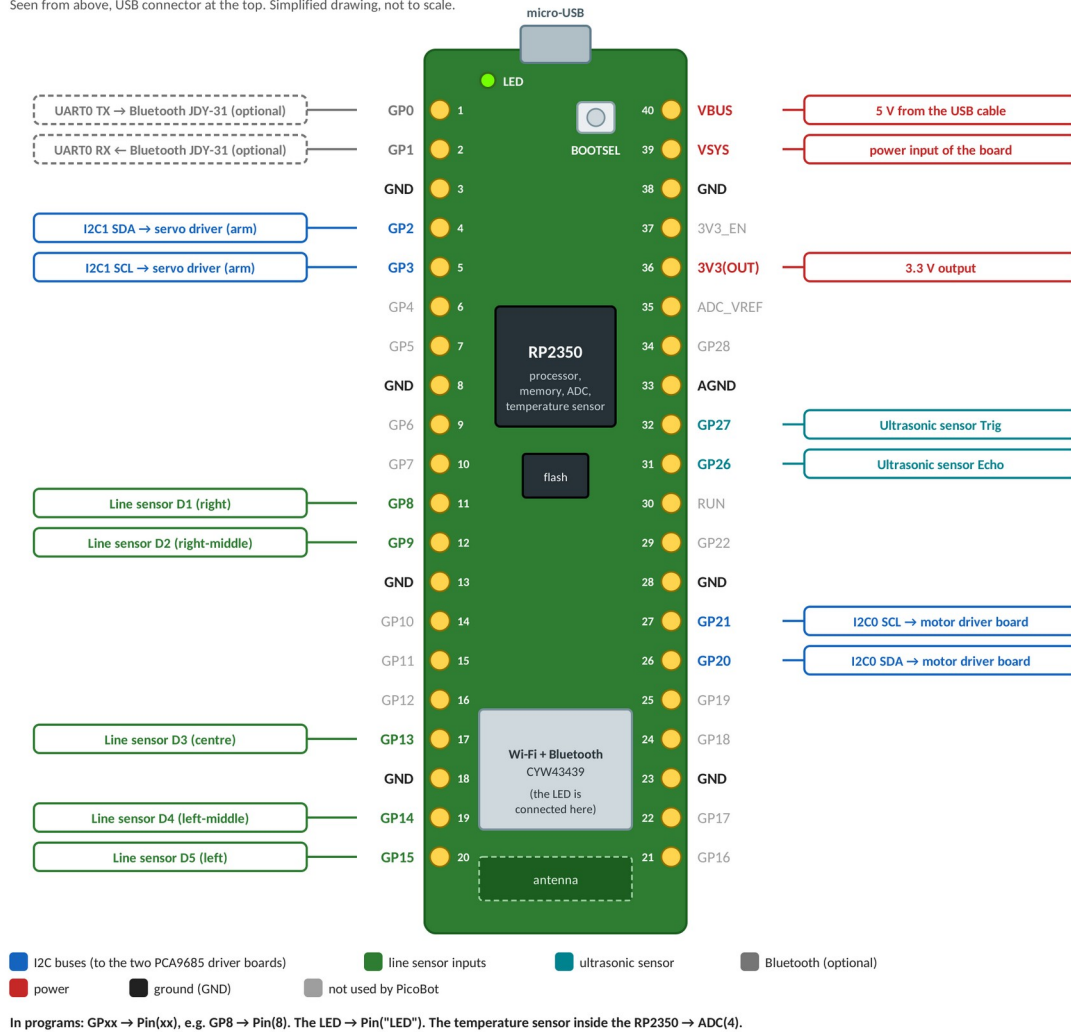
3.3 V operation. The classic HC-SR04 design is for 5 V. On PicoBot the sensor is supplied from the 3.3 V rail, so its Echo output is also 3.3 V and safe for the Pico (P2) — no level shifter is needed. The module supplied with the kit works reliably at 3.3 V. Replacement modules must be the **HC-SR04P** (or another 3.3 V-capable version, specified for 3–5.5 V); an original 5 V module may not work at 3.3 V and must never be supplied with 5 V on this robot, because its Echo output would then be 5 V.

In the software. The obstacle-avoidance program measures without blocking: a timer sends a trigger every 60 ms (the minimum cycle of the data sheet), and a pin interrupt on the rising and falling edge of Echo measures the pulse. Values outside 2–500 cm are treated as “no valid measurement”.

4.9 Pin map

Raspberry Pi Pico 2 W — the pins PicoBot uses

Seen from above, USB connector at the top. Simplified drawing, not to scale.



The pins of the Pico 2 W used on PicoBot (seen from above, USB connector at the top).

Pico pin	GPIO	Connected to	In MicroPython
1	GP0 (UART0 TX)	JDY-31 RXD (optional Bluetooth)	<code>UART(0, tx=Pin(0), rx=Pin(1))</code>
2	GP1 (UART0 RX)	JDY-31 TXD	
3	GND	JDY-31 GND	
4	GP2 (I2C1 SDA)	servo driver SDA	<code>I2C(1, sda=Pin(2), scl=Pin(3))</code>
5	GP3 (I2C1 SCL)	servo driver SCL	
11	GP8	Maker Line D1 (right)	<code>Pin(8, Pin.IN, Pin.PULL_UP)</code>
12	GP9	Maker Line D2 (right-middle)	<code>Pin(9, Pin.IN, Pin.PULL_UP)</code>
17	GP13	Maker Line D3 (centre)	<code>Pin(13, Pin.IN, Pin.PULL_UP)</code>
19	GP14	Maker Line D4 (left-middle)	<code>Pin(14, Pin.IN, Pin.PULL_UP)</code>
20	GP15	Maker Line D5 (left)	<code>Pin(15, Pin.IN, Pin.PULL_UP)</code>
26	GP20 (I2C0 SDA)	PCA9685 on the motor driver board (board track)	<code>I2C(0, sda=Pin(20), scl=Pin(21))</code>

Pico pin	GPIO	Connected to	In MicroPython
27	GP21 (I2C0 SCL)	PCA9685 on the motor driver board (board track)	
31	GP26	HC-SR04 Echo	Pin(26, Pin.IN)
32	GP27	HC-SR04 Trig	Pin(27, Pin.OUT)
36	3V3 (OUT)	WAGO 3 — 3.3 V rail	
39	VSYS	5 V from the motor board's regulator	
—	"LED"	green LED (on the wireless chip)	Pin("LED", Pin.OUT)

Free pins for extensions (R14): GP4, GP5, GP6, GP7, GP10, GP11, GP12, GP16, GP17, GP18, GP19, GP22 and **GP28** (the only free analog input) — 13 pins. GP4/GP5 and GP6/GP7 can form further I2C buses, so additional I2C sensors can be added without touching the existing ones. Before using a free pin, check that the motor board does not use it (its documentation lists only the I2C0 connection).

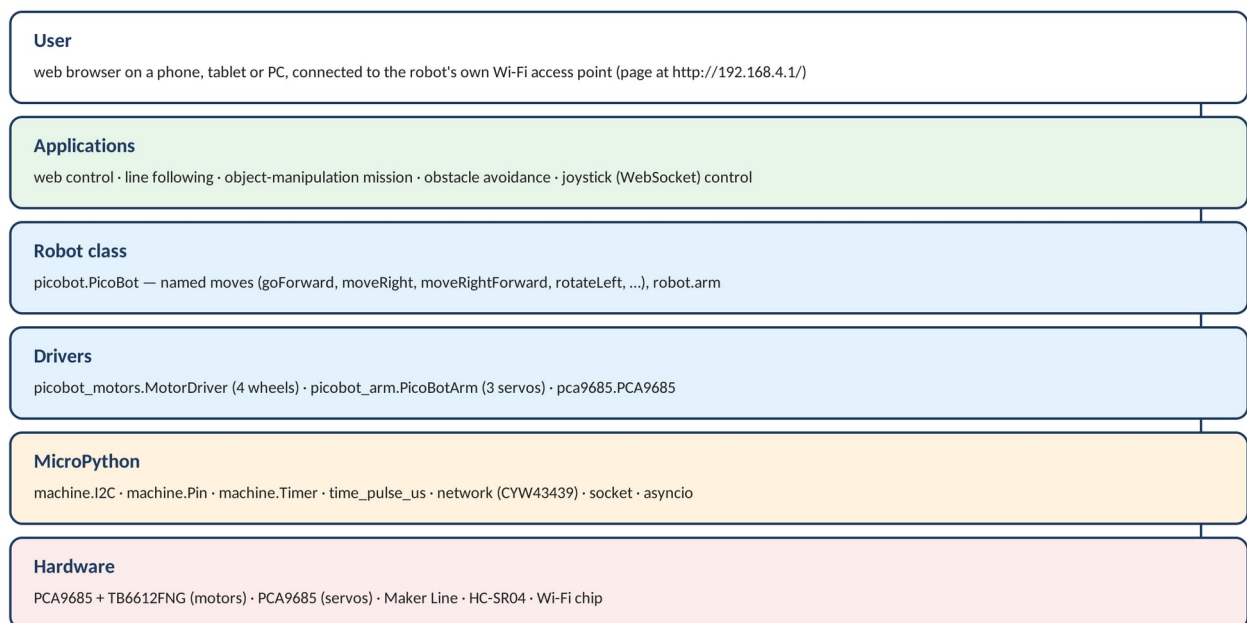
5. Control software on the robot

5.1 Platform

All software runs **on the Pico in MicroPython** (R1). A MicroPython program is a set of plain .py files in the Pico's flash memory; `main.py` starts automatically when the robot is switched on. During development the Pico is connected to a computer by USB and programmed with the Thonny editor, which also gives an interactive shell for testing single commands on the real robot. All programs are published under the MIT licence on <https://github.com/roboteamdev> (R14).

5.2 Software layers

Layers of the PicoBot software (all MicroPython, running on the Pico)



PicoBot software layers

Layers of the PicoBot software.

The software consists of these Python modules:

Module (file)	Layer	Job
pca9685.py	driver	access to the PCA9685 registers (frequency, pulse per channel) for the servo driver
picobot_motors.py — class MotorDriver	driver	the four wheel motors through the motor board's PCA9685 on I2C0 (GP20/GP21, 100 kHz, 50 Hz); TurnMotor(motor, direction, speed), MotorStop(), StopAllMotors(), MotorRun()
picobot_arm.py — class PicoBotArm	driver	the three servos through the servo driver on I2C1 (GP2/GP3, 50 Hz); control_servo(channel, angle), smooth_move_servo(...), init_servos() (all to 90°), reset_servos()
picobot.py — class PicoBot	robot	the named moves of section 3.4 (goForward(), goBackward(), moveRight(), moveLeft(), moveRightForward(), ..., rotateRight(), rotateLeft(), stopRobot(), hardStop()), default speed 50 %; robot.arm gives the arm
picobot_main.py	application	starts the Wi-Fi access point, serves the web page, receives and executes commands
web page (embedded in picobot_main.py)	user interface	movement buttons and arm sliders
main.py	start-up	starts the application when the robot is switched on

Why layers. Each layer hides the details of the one below. A beginner writes `robot.moveRight()` and sees the robot move sideways; an advanced student opens `picobot.py` and finds the wheel pattern of section 3.4; a specialist opens `picobot_motors.py` and sees the PCA9685 registers. All three work with the same, working robot (R7). New programs reuse the lower layers unchanged, which keeps every application small.

Two PCA9685 driver classes exist side by side: a register-level class inside `picobot_motors.py` (based on the manufacturer's example for the motor board) and the general `pca9685.py` used by the arm (third-party MIT-licensed code). Both drive the same chip type; keeping them separate means that the arm code and the motor code can be changed independently.

5.3 Motion control

Named moves. Each method of `PicoBot` sets the four motors to the pattern of section 3.4, with one speed for all active wheels. A move continues until the next command — the program decides **how long** a move lasts (time-based motion, section 3.6). In the four diagonal moves only two wheels are commanded; programs stop the robot before a diagonal move so that the other two wheels do not keep an earlier command.

Continuous mixing (joystick control). For manual driving (tasks 3 and 4) the joystick program converts three inputs — forward *f*, sideways *s*, rotation *r*, each $-100\ldots+100\%$ — with the kinematic equations of section 3.5 into four wheel values, scales them down together if one exceeds 100 %, and applies two practical corrections:

- a **dead band** of 5 %: very small stick movements give no motion, so the robot does not creep;

- a **minimum start value** of 20 %: below this duty cycle the motors cannot overcome friction, so the output is mapped to 20–100 % and every stick movement outside the dead band really moves the robot.

This allows every combination of the three motions at once — for example driving forward while turning, or moving diagonally while rotating — which the “master control” task is designed to show.

5.4 User interface and communication

Web control (beginner level). The robot starts a Wi-Fi access point and a small HTTP server. The web page shows buttons for the moves and sliders for the three servos. Pressing a button sends an HTTP request (e.g. /forward?; a slider sends /?servo_arm_slider=<angle>); the program parses it in its main loop and calls the matching PicoBot method. This is simple, works in every browser, and students can read the whole chain from button to motor in a few hundred lines.

WebSocket joystick control (advanced level). For fast manual driving the joystick program keeps a **WebSocket** connection open (ws://192.168.4.1/ws) and uses a compact text protocol: the browser sends drive frames D, seq, f, s, r at 20 Hz while the stick is used, arm frames A, seq, base, arm, claw, ARM/STOP frames, and the robot returns telemetry (mode, armed state, speeds, servo angles, distance, uptime) at 5 Hz. A sequence number in every frame lets the robot drop late or duplicated frames. The program runs on MicroPython’s `asyncio`, so receiving, driving, the arm planner and telemetry run as cooperating tasks.

Why a web interface. It needs no app installation and no extra hardware: any phone, tablet or PC with a browser is the remote control (R6). Because the robot is the access point, the system works in any classroom or competition hall without a network.

5.5 Programs for the four reference tasks

Task	Program	How it works
1 — Line tracking with number of laps	line following / lap counting	every 50 ms the 5 sensor values are read; the active sensors give a weighted position (weights +2 ... -2); the position selects one of seven steering actions (forward, slight / mild / hard left or right) by slowing the wheels on one side (ratios 0.9 / 0.75 / 0.6); if the line is lost, the robot keeps the last direction for a grace time and then searches. When all five sensors see black, the robot is on the marker: each crossing is counted once (with a 1 s debounce), and the robot stops after the set number of laps. Speed, steering ratios and the number of laps can be changed on the web page

Task	Program	How it works
2 — Manipulation of an object	object-manipulation mission	a non-blocking state machine (modes IDLE → OUTBOUND → ARM_SEQ → RETURNING → DONE, timers every 50 ms): follow the line to the marker; turn the base 45° to one side; lower the arm, close the gripper, lift; turn the base 45° to the other side; lower, open, lift to the transport position; drive back about 10 cm; turn 180°; follow the line back to the start marker. All angles and times are parameters (e.g. 600 ms for each servo to settle, about 600 ms for 10 cm back, about 1000 ms for 180°), tuned per robot
3 — Master control	joystick control	manual WebSocket control with mecanum mixing (section 5.3): diagonal passes and sideways parking
4 — Separate garbage collection	joystick control / web control	manual driving plus the arm sliders: grip, carry and drop the foam objects

A further program, **obstacle avoidance**, uses the ultrasonic sensor: the robot drives forward, and when an obstacle is closer than a set distance it moves **sideways** around it (strafing — only possible with mecanum wheels), then continues. It shows the use of interrupts and timers for non-blocking measurement (section 4.8.2).

Why state machines and timers. A robot must do several things “at the same time”: watch the sensors, drive, move the arm and answer the web page. A program that waits with `sleep()` cannot react while it waits. The PicoBot programs therefore call short functions from timers every 50 ms and keep the progress of the task in a state variable; each call does a small step and returns. This keeps the robot responsive and makes the mission logic readable as a list of states.

5.6 Safety functions in software

Function	What it does	Why
Arm to home at start	creating <code>PicoBot()</code> or <code>PicoBotArm()</code> moves all servos to 90°	the arm starts from a known pose; the program knows the servo positions
Safe angle limits	arm and gripper 40°–140°, base 0°–180°; the joystick firmware clamps every requested angle	prevents collisions with the chassis and servo stall (section 3.7)
Smooth servo moves	<code>smooth_move_servo()</code> and the joystick's servo speed limits (degrees per second)	no jerks that stress the gears; the gripper closes gently
Motors stopped at start and on STOP	<code>StopAllMotors()</code> switches all bridge outputs off	a known, safe state
Dead-man timer (joystick)	if no drive frame arrives within the set time, the motors are switched off; STOP and disarm commands are always accepted	a lost Wi-Fi connection must not leave the robot driving (see 7.2)

Function	What it does	Why
Arm/disarm (joystick)	the motors respond only after an explicit “arm” command	the robot cannot start by accident when the page is opened

Configuration note. The dead-man time is set in `config.py` of the joystick program. Use **250 ms** for classes and competitions; longer values (e.g. 2000 ms) are only for testing (section 8).

6. Safety design

Hazard	Design measure	Operating rule
Electric shock	the robot uses only extra-low voltage (at most 8.4 V)	—
Short circuit of the battery (unprotected cells)	on/off switch in the holder; WAGO lever connectors hold wires firmly; the Pololu regulator has short-circuit protection	wiring only with the switch off; no bare wire ends; never short + and –
Over-discharge of the cells	optional voltmeter; operating limit 7.0 V (well above the 6.0 V cell limit)	recharge at 7.0 V; keep cells as labelled pairs; charge each cell in its own bay
Charging fire risk	charging outside the robot with an 18650 charger	charge on a non-flammable surface, not unattended; do not use damaged or swollen cells
Reversed supply to the servo regulator	Pololu D30V30F5 reverse-voltage protection (up to 40 V)	check + and – before switching on
5 V on a Pico pin	all sensors and the servo driver logic on the 3.3 V rail (P2); 3.3 V-capable ultrasonic sensor	never connect VSYS or VBUS to the 3.3 V rail
Pico reset by current peaks	three separate power branches (P1)	—
Pinched fingers in the arm or gripper	small servos with limited torque (about 2 kg-cm); arm to home at start; safe angle limits	hands away from the arm when a program starts
Robot drives away uncontrolled	dead-man timer and STOP command in the joystick control; STOP button on the web pages	test motor programs with the wheels lifted
Servo overheating, stripped gears	safe angle ranges; smooth moves; metal-gear servos as replacement	choose a “closed” gripper angle that just holds the object
Soldering	only one soldering step (header pins of the regulator), done once by a teacher	soldering under supervision, with ventilation and eye protection

7. Verification

7.1 Requirements check

Requirement	How the design meets it	Status
R1 Pico W / compatible, Python	Pico 2 W (Pico W compatible), MicroPython	met

Requirement	How the design meets it	Status
R2 Omnidirectional drive	4 mecanum wheels in X pattern; 8 named moves + rotation; continuous mixing	met; all base movements verified in tests
R3 Arm with gripper	2-joint arm + gripper, base $\pm 90^\circ$ from home	met; gripping, transport and placement of an object verified in tests
R4 Line sensing and markers	5-channel Maker Line; marker = all five sensors	met
R5 Distance sensing	HC-SR04, 2–400 cm	met
R6 Manual control without extra hardware	Wi-Fi access point, web page, WebSocket joystick	met
R7 Programmable at several levels	layered library (section 5.2)	met
R8 Battery operation	2 × 18650, about 19–27 Wh	met; running time see section 9
R9 Assembly without special skills	plug-in Pico, screw terminals, WAGO, jumpers, Command strips; one soldering step	met
R10 Robust, repairable	few moving parts, modular boards, servo limits	met by design (section 3.8)
R11 Safe	section 6	met
R12 $\leq 30 \times 30 \times 30$ cm, ≤ 2 kg	25.5 × 16.5 × 22.2 cm; about 1 kg (estimate)	met
R13 Low cost, standard parts	commercial kit + standard modules (Appendix A)	met
R14 Open, extensible	MIT/CC BY; 13 free GPIO, 13 free servo channels, free analog input	met

7.2 Findings of the first tests and the design response

The first tests with the web-controlled robot covered (1) control of all movements of the base and the arm, and (2) gripping an object, transporting it and placing it in a marked target area. Both confirmed the basic functions. The shortcomings found in these tests and the design answers are:

Shortcoming found in the tests	Design response
Commands are not executed instantly because of communication delays	WebSocket joystick control: a permanently open connection instead of one HTTP request per command; drive frames at 20 Hz
Wi-Fi interference can break the link; the robot may keep its last command	dead-man timer (motors off when frames stop), STOP always accepted, arm/disarm
Buttons and sliders need many manual actions; slow in emergencies	analogue joystick with continuous mixing of forward, sideways and rotation; one STOP command
Excessive tightening of the gripper can make the servo fail	safe angle ranges clamped in the software; gripper “closed” angle chosen to just hold the object; metal-gear servos as replacement

Shortcoming found in the tests	Design response
No automatic correction of position errors	autonomous programs use the line sensor and markers as absolute references; encoders are a possible extension (section 8)
No smooth transitions — faster mechanical wear	<code>smooth_move_servo()</code> ; servo speed limits and motor dead band / minimum start value in the joystick program

8. Known limitations and recommended improvements

The items below are **recommendations** for builders who want to improve the robot. They are not part of the standard design and have not yet been tested on PicoBot.

#	Limitation	Recommendation
1	Motor PWM at 50 Hz : the motors run in audible steps at low speed; low-speed control is coarse	raise the motor-board PCA9685 frequency (allowed up to about 1.5 kHz; the servo chip stays at 50 Hz) and check the effect on low-speed driving
2	At 100 % duty the TT motors get up to 8.4 V, above their 3–6 V rating	limit the maximum duty in the library to about 70 % for long runs, or accept the extra speed for short competition runs
3	No wheel encoders: distances and turns are time-based and depend on battery and floor	add encoders, or calibrate times per robot and per battery state
4	The Pico cannot read the battery voltage	add a voltage divider (e.g. 1:3) from the battery to the free analog input GP28; show the voltage on the web page and stop at 7.0 V
5	No fuse in the battery line; unprotected cells	add a small automotive blade fuse or a resettable fuse (about 3–5 A) between the holder and WAGO 1
6	The motor library writes the PCA9685 registers one by one (about 15 ms for four wheels)	use auto-increment block writes (as the joystick program does) in the main library too
7	Diagonal moves command only two wheels; the other two keep an earlier command	let the diagonal methods also stop the two unused wheels
8	The joystick dead-man time is set to 2000 ms (testing value)	use 250 ms for classes and competitions
9	The JDY-31 Bluetooth module is supplied from 3.3 V, below the 3.6–5 V specification of the breakout	check operation at 3.3 V before Bluetooth is used, or leave the module out
10	Every robot running the same program has the same Wi-Fi name	give each robot a unique network name (e.g. with its number)
11	Plastic-gear servos can strip under load	use metal-gear MG90S servos as standard replacement
12	The line sensor's analog position output is not used	connect it to GP28 for a continuous position value (if GP28 is not used for item 4)

9. Values to measure on a built robot

These values depend on the individual robot, its cells and the floor. This document gives them only as estimates or not at all; measure them on your robot when you need exact figures.

#	Quantity	How to measure	Estimate in this document
1	Mass with cells	kitchen scale	about 1 kg (limit 2 kg)
2	Wheel diameter, track width, wheelbase	ruler / calliper	$\varnothing \approx 6$ cm; track ≈ 13.5 cm; wheelbase 10.2 cm
3	Forward, sideways and diagonal speed at 50 % and 100 %	time over 1 m with full cells	≤ 0.63 m/s forward at 6 V
4	Time for a 90°, 180°, 360° turn	stopwatch or video	≤ 300 °/s
5	Current of the whole robot: standing, driving, arm moving, motors stalled	multimeter in series at the battery holder	1–2 A driving
6	Running time with a 2600 mAh pair from 8.4 V to 7.0 V	test drive with a timer	1–2.5 h continuous driving
7	Current of the 3.3 V rail	multimeter in the WAGO 3 feed	well below 300 mA
8	Reach and lifting height of the gripper; largest safe object mass	ruler, test weights	a few tens of grams
9	JDY-31 operation at 3.3 V	pairing test	—

10. Hardware revisions

Stage	Configuration
Starting point	commercial 4WD mecanum robot-arm kit with Arduino Uno, expansion board, 3-channel line sensor, Bluetooth module, infrared remote
First PicoBot prototypes	Raspberry Pi Pico 2 W on the Waveshare Pico-Motor-Driver; PCA9685 servo driver; three MG90S servos; ultrasonic sensor; 2 × 18650 cells (2600 mAh); Pololu D24V25F5 (5 V, 2.5 A) step-down regulator and an adjustable buck-boost converter for the PWM controller; web control over the robot's own access point
Rev2	Pololu D30V30F5 (5 V, 3.4 A); Cytron Maker Line 5-channel line sensor; WAGO power distribution; optional voltmeter; DuPont jumpers; Command-strip mounting
Rev3 (current, described in this document)	the motor board is supplied directly from the battery (WAGO 1), not from the 5 V regulator — the servo regulator now feeds only the servos (principle P1); corrected Bluetooth TX/RX wiring
Software	from button/slider web control to line following, object-manipulation mission, obstacle avoidance, lap counting and WebSocket joystick control with safety functions

Appendix A. Bill of materials (Rev3)

Part	Qty per robot	Job	Notes
Tscinbuny 4WD mecanum robot-arm car kit (for Arduino)	1	chassis, 4 TT gear motors with mecanum wheels, arm with 3 servos, HC-SR04, JDY-31, 2 × 18650 holder with switch	kit's Arduino boards, 3-channel line sensor and IR remote not used
Raspberry Pi Pico 2 WH (headers soldered) — or Pico 2 W; Pico W / WH also works	1	controller, Wi-Fi	WH version avoids soldering
Waveshare Pico-Motor-Driver (19764)	1	base board, 4 motor channels, 5 V for the Pico	
PCA9685 16-channel 12-bit PWM servo driver board (I2C)	1	arm servos	
Pololu D30V30F5 step-down regulator, 5 V, 3.4 A	1	servo supply	header pins soldered once
Digital voltmeter, 2-wire, 2.5–40 V	0–1	battery voltage display	optional
Cytron Maker Line line sensor	1	line and marker detection	
DuPont jumper wires F-F 10 cm, M-F 10 cm, F-F 20 cm	1 set each	signal and low-current wiring	
WAGO 221-415 lever connector, 5-way	2 (+ spare)	WAGO 1 (+), WAGO 2 (GND); a third for the 3.3 V rail	
WAGO 221 lever connector, 2-way	4	extending short wires	
18650 Li-ion cells, 2600–3600 mAh, unprotected, known brand	2 (+ 1–2 spare pairs)	power	over 3600 mAh claims are usually fake
18650 charger, 4 or 6 bays (mains-powered preferred)	per class	charging	
3M Command strips (hook-and-loop)	about 8 pairs	removable mounting of the boards	

Appendix B. Wiring list (Rev3)

Power

From	To	Voltage
Battery holder + (after switch)	WAGO 1 (battery +)	6.0–8.4 V
Battery holder –	WAGO 2 (battery – = common ground)	0 V

From	To	Voltage
WAGO 1 / WAGO 2	motor driver board power terminal + / - (VIN)	6.0–8.4 V
WAGO 1 / WAGO 2	Pololu D30V30F5 VIN / GND	6.0–8.4 V
Pololu VOUT / GND	servo driver external power terminal + / - (V+)	5 V
Motor board header, Pico pin 36 = 3V3 (OUT)	WAGO 3 (3.3 V rail)	3.3 V
WAGO 3	Maker Line VCC, HC-SR04 VCC, servo driver VCC (logic), JDY-31 VCC	3.3 V
All GND of the sensors, the servo driver logic and the JDY-31	WAGO 2 or a Pico GND pin	0 V

Motors (screw terminals of the motor board; red = +, black = -)

Terminal	Motor
A1 / A2	M1 front-left
B1 / B2	M2 back-left
C1 / C2	M3 front-right
D1 / D2	M4 back-right

If a wheel turns backwards in the single-wheel test, the two wires of that motor are swapped.

Signals

Pico pin (GPIO)	To
1 (GP0, UART0 TX)	JDY-31 RXD
2 (GP1, UART0 RX)	JDY-31 TXD
3 (GND)	JDY-31 GND
4 (GP2, I2C1 SDA)	servo driver SDA
5 (GP3, I2C1 SCL)	servo driver SCL
11 (GP8)	Maker Line D1 (right)
12 (GP9)	Maker Line D2 (right-middle)
17 (GP13)	Maker Line D3 (centre)
19 (GP14)	Maker Line D4 (left-middle)
20 (GP15)	Maker Line D5 (left)
26 / 27 (GP20 / GP21, I2C0)	PCA9685 of the motor board (on the board)
31 (GP26)	HC-SR04 Echo
32 (GP27)	HC-SR04 Trig

Servos (servo driver headers): channel 0 = base, 1 = arm, 2 = gripper; brown → GND (black pin), red → V+ (red pin), orange → signal (yellow pin). Each servo is set to 90° by a program before its horn is fitted.

If you work from older wiring pictures or lists: the motor board is fed from WAGO 1 (Rev2 fed it from the Pololu VOUT); Pico pin 1 is GP0 = **TX** and pin 2 is GP1 = **RX** (some lists swap the labels); some older connection pictures show the line sensor on GP11/GP12 and the motor I2C lines swapped — correct are **GP8/GP9** and **GP20 = SDA, GP21 = SCL**.

Appendix C. References

PicoBot

- PicoBot program code (MIT licence): <https://github.com/robosteamdev> — repositories picobot-setup, picobot-web-control, picobot-line_following, picobot-lap-counting, picobot-mission-control, picobot-obstacle-avoidance, picobot-joystick-control, PicoBot.
- PicoBot assembly video: https://www.youtube.com/watch?v=kURZ2_49MeE
- ROBO STEAM ACADEMY project website: <https://robosteam.eu/>

Data sheets and manufacturer information

- Raspberry Pi Pico 2 W data sheet: <https://datasheets.raspberrypi.com/picow/pico-2-w-datasheet.pdf>
- Waveshare Pico-Motor-Driver: <https://www.waveshare.com/wiki/Pico-Motor-Driver>
- Toshiba TB6612FNG (summary and truth table): <https://www.pololu.com/product/713>, <https://learn.sparkfun.com/tutorials/tb6612fng-hookup-guide/all>
- NXP PCA9685: <https://www.nxp.com/products/PCA9685>
- Pololu D30V30F5: <https://www.pololu.com/product/4892>
- Cytron Maker Line data sheet: <https://cdn.robotshop.com/media/c/cyt/rb-cyt-274/pdf/cytron-maker-line-line-sensor-datasheet.pdf>
- HC-SR04 user manual: https://www-users.york.ac.uk/~mjf5/roach/Datasheets/HCSR04_Manual.pdf; 3.3 V version: https://cdn.sparkfun.com/assets/a/8/9/c/a/HC-SR04-33_Datasheet.pdf
- SG90 micro servo: <https://bit.nkust.edu.tw/~csshie/teach/111A/mc/datasheet/SG90.pdf>; measured currents: <https://protosupplies.com/product/servo-motor-micro-sg90/>
- MG90S metal-gear servo: <https://components101.com/motors/mg90s-metal-gear-servo-motor>
- TT gear motor 1:48 (typical data): <https://thepihut.com/products/dc-gearbox-motor-tt-motor-200rpm-3-to-6vdc>
- JDY-31 Bluetooth module: <https://martyncurrey.com/jdy-31-spp-bluetooth-module>

Background

- A. Gferrer: *Geometry and kinematics of the Mecanum wheel*, Computer Aided Geometric Design 25 (2008) 784–791.
- B. Siciliano, O. Khatib (eds.): *Springer Handbook of Robotics*, Springer, 2008.

Credits and licence



Co-funded by
the European Union



PicoBot Robot Design — the design of the PicoBot educational robot (mechanics, electronics, control software), developed in the Erasmus+ project **ROBO STEAM ACADEMY** (KA220-VET — Cooperation partnerships in vocational education and training, project number KA220-VET-7CF4F308, 2024–2027). Project website: <https://robosteam.eu/>

Partners: National STEM Center (coordinator, Bulgaria, <https://stem.mon.bg/>) · Institute of Information and Communication Technologies, Bulgarian Academy of Sciences (Bulgaria, <https://www.iict.bas.bg/>) ·

Academy Nikola Tesla (Bulgaria, <https://ant.bg/>) · Technical University of Košice (Slovakia, <https://www.tuke.sk/>) · Stredná priemyselná škola elektrotechnická Prešov (Slovakia, <https://spseadlerka.edupage.org/>) · National Association of Public Librarians and Libraries in Romania – ANBPR (Romania, <https://anbpr.org.ro/>) · Liceul Tehnologic Cezar Nicolau (Romania, <https://www.cezar-nicolau.ro/>).

Licence: texts, images and 3D-print files **CC BY 4.0** (Creative Commons Attribution 4.0 International, <https://creativecommons.org/licenses/by/4.0/>); program code **MIT licence**. You may share and adapt the materials, also commercially, if you credit “ROBO STEAM ACADEMY, Erasmus+ project KA220-VET-7CF4F308” and indicate changes. Third-party images, code and parts keep their own licences and are credited where they are used.

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the Human Resource Development Centre (HRDC). Neither the European Union nor the granting authority can be held responsible for them.